Meaning as a Path

VSS · VSA Semantic Space: a verifiable semantic layer between language and action

How a sentence becomes a constellation, what can be read off its chart, and what you can do with that

---

Contents

One-page summary

1. The idea we started from

1.1 What we mean by meaning
1.2 Who takes part
1.3 Where we stand

2. The sky, the probe and the pilot

2.1 A point in the sky: coordinates with thousands of dials
2.2 Why two points chosen at random are unrelated
2.3 The probe: a single operation
2.4 The word's star
2.5 The sentence's chart: how a constellation is drawn
2.6 The legend and the domains
2.7 The scanner: how the chart is read

3. Reading a sentence

3.1 The step
3.2 What it means for two sentences to "end up in the same place"
3.3 Reading the chart: the frame
3.4 Sentences within sentences: the legend is relative
3.5 The beacons on the chart: how a word knows which domain to go to
3.6 Three drawings for the same word

4. Why two sentences of different lengths can mean the same thing

Length is a property of the route, not of the chart
Function words move the probe, but do not enter the frame
Different words, the same destination
One example traced, and one that does not match
When length does matter, and what the chart does then

5. The same word in several worlds: polysemy, fields of knowledge, ontologies

The star is the word, not the sense
When the kind of the word is ambiguous
The world is a first-class object
Several worlds at once
Do we discover ontologies?
The same words, different theories

6. How the pilot learns

6.1 What he receives: the world's constellation
6.2 Why this forces meaning
6.3 What is learned, and how little

7. What remains on the chart after a sentence, and what it is good for
8. Phenomena you can follow on the chart
The empty domain, and why “empty” is harder than it looks
The crowded domain
Stars outside the legend, or how the pilot knows a sentence makes no sense to it
The broken key
Noise, and why the chart has a capacity
Negation as a star, and where its scope lives
Ambiguity held open
The level never seen, and the key that does not exist
A new construction: a missing beacon
Structure announced afterwards
Order, and what commuting keys cannot tell apart
Which verb? The predicate is a domain too
Several events, and the same name twice
Three kinds of “close”
What the pilot does not remember
The pronoun: a borrowed star
Why the word stars are random, and must stay that way
9. Where it leads
9.1 Assistants allowed to say “I don’t know”
9.2 A verifier beside the large models
9.3 Formalising a field for symbolic reasoning
9.4 A boundary that learns
9.5 A world model you can read
10. Robotics: a semantic interlock
What it looks like
Why it is cheap, and why it can be hardware
What it does when it is not sure
What decides how far this goes
11. Enterprise and highly regulated industries
Agentic systems: intent read without a language model
Regulated industries: documents that must be read exactly
Where large models come in, and where they do not
What sets the pace
12. Further on: documents, novelty, bias, smoothing
13. For the technically inclined: the story, translated
The family
Atoms and compounds
What the model computes
The world and the objective
Reading back and the diagnostics
What is standard, what is ours
14. What the sky can be made of: representation variants, sparse codes and the brain
The same three operations, several materials
Why we chose phases
The sparse code: the same sky, written in neurons
Why the brain’s code is believed to be sparse
Where the innovation is, and where the engineering is
15. The parallel with A Thousand Brains
What the theory says, in five sentences
The dictionary
What we took literally

Where the vote is
Why only three
Where we part ways, and why

16. Alignment and the problems around it: what can be said in engineering terms

The basic idea: the legend as a boundary
The problems, one by one
What it does not solve
Summary

17. What it is, and what it is not

What it is
What it compares against
What it is not
What could change the message
Limits

One-page summary

The thesis. The meaning of a sentence, in a given domain, is the set of answers it gives to a short, fixed list of that domain's questions (who? what? to whom? when? under which condition?), including the answer "it does not say". Two sentences mean the same thing when they give the same answers, however differently they sound. We call the translation of a sentence into such verifiable answers *the formalisation of language*.

The mechanism. A small reader, with no knowledge about the world, walks through the sentence word by word as along a route in a space of thousands of dimensions: each kind of word is a learned jump; each word is drawn where the route has reached; different routes that reach the same named places give the same frame. The named places (the roles) come from the domain; the route is learned from examples with their frames; nothing else is learned. Everything the reader knows fits in a megabyte; reading costs microseconds on an ordinary processor; the same input always gives the same output.

What has been shown so far, on constructed worlds and on a real set of questions. Different formulations of the same event reach the same frame. The reader composes correctly at a nesting depth it never saw in training, where a neural network with the same input guesses. New words are read without retraining. Logical operators (negation, modality, conditionals) are read with their scope correct, without hand-written rules. On real questions, with the same input and no external knowledge, the role structure is learned better than by a small transformer or a lexical method, at every data size tried. And what was not understood shows on the chart (an empty domain, a crowded domain, an unexplained remainder), instead of being covered by a fluent answer.

What it is not. Not a language model: it does not generate, does not converse, holds no facts. It does not replace large models on open text. Its place is beside them: the verifiable layer between language and action, the one that says what was understood, what was not, and where, and through which everything that reaches execution or the database must pass.

Where it leads. Assistants on closed domains that are allowed to say "I don't know"; a verifier of the formalisations made by large models; a semantic interlock for robots, cheap and implementable in hardware; the exact reading of procedures, regulations and reports in regulated industries; a verifiable interface for the alignment of agentic systems. Each is described with its mechanism and with the measure that sets its pace.

State and pace. The signals are good and of the kind that matters; what remains is, for the most part, engineering and adjustments of the model, with incremental steps and criteria written in advance. The exact state of every measurement is kept up to date in a single public file: https://github.com/ml-circuits-research/vsa-semantic-space/blob/main/STATUS.md

---

1. The idea we started from

When you read a sentence, you do not gather the words into a sack. You travel. Each word moves you somewhere, and what you are left with at the end is not the list of words, but the place you reached and what you left along the way. “Ion gives Ana a book” and “Ana receives a book from Ion” are two different routes that end in the same place. That is why they mean the same thing.

We took this image seriously and built it. To explain it without cheating, we describe below an imaginary story, a sky full of stars, a probe, a pilot, and the world the sentences talk about, which behaves exactly like the mathematics behind the system: every statement in the story is also true of the real system, and no piece of it does anything the real system does not do. At the end, a table ties each piece to its technical name.

1.1 What we mean by meaning

Before the story, a word about the word “meaning”, because we use it in a particular sense, narrower than the usual one and, for that very reason, measurable.

A sentence describes an event: someone gave something to someone, a supplier delivered some boxes somewhere, a teacher holds a class in a room at a given hour. About such an event one can ask questions: who? what? to whom? where? when? how much? Not any number of them and not any question: for a given domain, the list of questions that make sense is short and fixed. A lending ledger never asks “at what temperature?”; a warehouse never asks “who liked it?”. To know what a sentence means, in this document, is two things: to know *which questions can be asked* about the event it describes, and to know, for each of them, *what answer the sentence gives*, including the answer “it does not say”. An answer may be a word, it may be the negation of a word (“not Ion”), and to “what does Maria say?” it may be a whole event, with its own questions.

This definition has three consequences we will use throughout. First, meaning is relative to a domain: the same sentence means something different to the warehouse and to the timetable, because their lists of questions differ; there is no “meaning in itself”, there is meaning relative to a list of questions. Second, two sentences mean the same thing exactly when they give the same answers to the same questions, however differently they sound. Third, “I did not understand” becomes something precise: a question left unanswered although the sentence should have answered it, an answer in which two candidates compete, a piece of the sentence that answers no question on the list, or a question the sentence raises that the list does not contain.

Answers have two sources, and we keep them apart. The sentence says what it says: “Ion gives Ana a book” answers who, what and to whom, and is silent on when. The domain’s database knows the rest, or contradicts: that the loan was on Tuesday, that Ion owns no book. The system described here does only the first part: it turns the sentence into a complete set of answers to the domain’s questions, with the gaps marked, in a form that a database or a reasoning engine can check and complete. We call this transformation *the formalisation of language*: not the formalisation of grammar, but the translation of what is

said into verifiable answers to questions known in advance. The rest of the document is about how this is done without the system holding any knowledge base about the world.

1.2 Who takes part

Four participants, and it is important not to mix them up.

The sky. A single universe, almost empty. A point in the sky is given by *coordinates*, and that is all: there is nothing in it besides coordinates. Some points have been given a name or marked by someone; those we call *stars*. A star is not an object; it is a point someone has written down. We will meet three kinds of stars: the words' stars, the world's named stars, and the stars the pilot draws on a sentence's chart. The sky is not the world the sentences talk about and knows nothing about it.

The probe. The only thing that moves through the sky. It knows how to do two things and nothing else: to *jump* from one point to another, when it is given a key, and to *report* the coordinates of the point it has reached. At the start of every sentence it is at the *origin*, the point with all coordinates zero.

The world. The domain the sentences talk about, and the only place in the story where facts exist. Three examples, to make it concrete:

- A *warehouse*. Its sentences sound like this: "Supplier Alfa delivered 40 boxes to aisle B on Tuesday." The warehouse wants to know, from any sentence: *who delivered? what? how much? where? when?*
- A *school timetable*. "Mrs Pop teaches physics in room 12 on Tuesday at 10." The timetable asks: *who? which course? in which room? when?*
- A *lending ledger* between friends. "Ion gives Ana a book." The ledger asks: *who gives? what? to whom?*

Each world therefore has a short list of questions, always the same for all its sentences. A sentence answers some and leaves others empty ("Ion gives Ana a book" does not say *when*). The thing a sentence describes (a delivery, a lesson, the handing over of a book) we will call an *event*; the word means nothing more than "what happened, in this sentence". The world lives outside the pilot, for instance in a database: it knows that Alfa is a supplier and that aisle B exists. The pilot knows none of this. A world has things of its own: the legend, the kinds of words, the route book learned on its sentences and the database; chapter 5 takes them in turn. In the rest of the document we use the lending ledger as the main example, because it is the smallest.

The pilot. A stranger. Think of him as a cartographer come from very far away, who does not understand a word of our language: he does not know who Ion is, what a book is, whether Ion really gave it to Ana, and has no way of finding out, because he never sets foot in the world. He does not move; he stays at the *base*, outside the sky, and commands the probe. What he knows, and knows very well, is a single craft, learned from thousands of sentences: to recognize the *kind* of a word (person's name, verb, preposition…) and to know what jump that kind calls for. His purpose is one alone: to turn each sentence into a chart of the sky that the world can read, that is, to find out, for each word, which *domain* of

the sky it must reach, without understanding what the word means. That he does not understand is not a shortcoming of the story; it is exactly what we want to show: meaning can be reached with no knowledge of the world, from the shape of the path alone. The pilot keeps three documents, and the whole story is about them:

- *The route book*: what jump each kind of word calls for (person's name, verb, preposition…). It is the only thing the pilot has ever learned, and the only thing he will learn.
- *The legend*: the world's list of questions, with a *named star* beside each question. For the lending ledger: "who gives?" has *the agent's star*; "what?" has *the object's star*; "to whom?" has *the recipient's star*. For the warehouse: the supplier's star, the goods' star, the quantity star, the place star, the time star. Each question is a *role*, and its star defines a *domain* of the sky, the agent's domain, the quantity domain, which the word that answers the question must reach (what exactly a domain is, in 2.6). In this book the word "domain" means only that; for a field of knowledge or activity we say "world". The named stars are drawn at random once, by whoever builds the world; nobody learns them. The legend is small: a few dozen stars for a whole field of activity. (It usually also has a star for the action itself, "given" or "lent"; chapter 8 says why it is needed.)
- *The sentence's chart*: the document the pilot *builds* while reading, word by word, and which at the end is the sentence's *meaning*. A chart of the sky, empty at first; at each word the pilot draws a star on it: the word's star, shifted by the coordinates the probe reported. At the end, the chart is a *constellation*; the pilot reads it through the legend and starts a new one.

The meaning of a sentence is therefore something very concrete: *which words reached which domains*. The names of the domains come from the legend (from the world), the path comes from the route book (learned), and "who reached where" comes from the chart (built while reading). Nothing ever lights up in the sky; everything that remains is on paper, at the base.

The rest of the chapter describes the pieces in the order in which they are used: the coordinates of a point, the probe's jump, the word's star, the chart and the constellation, the legend and the domains, and the scanner with which the chart is read.

1.3 Where we stand

This document describes a system that has been built and measured, on constructed worlds and on a real set of questions, and a direction that goes much further than what has been measured. We say once, here, how things stand, so that the rest can be read without a reservation in every paragraph.

The signals are good, and they are of the kind that matters. Different formulations of the same event reach the same frame. The system composes correctly at a depth it never saw in training, where a neural network with the same input guesses. New words are read without retraining. On real questions typed by people, with the same input and no external knowledge, it learns the structure better than a small transformer and better than a lexical

method, at every data size from a thousand examples up. Logical operators, with their scope, are read without hand-written rules. And what was not understood shows on the chart, instead of being covered by a fluent answer. None of these results required a large model: training takes minutes on a single small accelerator, and reading runs on an ordinary processor.

What remains is, for the most part, engineering and adjustments of the model; a few extensions (the order of operators, the distinction between set and alternative) require representation choices, described in their place; none is a known barrier. Every piece not yet in place has a precise mechanism, described in its chapter, and a measurement with its criterion written in advance. How far this goes depends on effort, and the effort is incremental: each step adds a domain, a legend or a measurement, without changing what lies underneath. That is why the chapters that follow explain *why* such a system is useful and *how* it is built, and do not keep the score of the measurements. The score is kept in a single file, up to date, with the criteria and the logs of every experiment, in the project's repository:

https://github.com/ml-circuits-research/vsa-semantic-space/blob/main/STATUS.md

The book's convention is simple: it describes the architecture and the direction, including proposed capabilities, and says locally when an application is a design ("could", "we envision"); the file says, at any moment, what is built, what is in progress and what is only a direction. Where the two differ, the file is right.

2. The sky, the probe and the pilot

2.1 A point in the sky: coordinates with thousands of dials

The coordinates of a point are a *code*, as on a combination lock: a string of dials, a few thousand of them, each of which can be turned anywhere on its circle, from 0 to 360 degrees. One particular position of all the dials at once is a point in the sky. Another position is another point.

So that we can write examples, let us pretend there are only four dials. Then a point looks like this: (10°, 200°, 95°, 300°). Another: (12°, 198°, 97°, 301°). A third: (180°, 20°, 275°, 120°).

Two points are *close* when, dial by dial, they look about the same: the first two above are close. They are *unrelated* when the dials do not match: the third has nothing in common with the first two. That is the whole geometry of the sky: not distances in light-years, but agreement or disagreement between dials, counted over thousands of dials. Between two points there are, nevertheless, intermediate points, and we will need them: a point whose dials sit halfway between those of the two is half close to each.

And one more thing, so that no doubt remains: the sky is empty. There is nothing in it of its own, no world, no words, no beaten paths. There are only points, and a point is a code. Stars are points someone has written down somewhere, in the catalogue, in the legend or on a chart; the sky itself remembers nothing.

2.2 Why two points chosen at random are unrelated

Here is the property on which everything else rests. Turn all the dials at random, twice, and compare the two codes. At one particular dial, they match by chance; at another, they do not; the matches and mismatches cancel out. Over thousands of dials, the total agreement is almost exactly zero. Two points chosen at random are, with practical certainty, *unrelated*: not the farthest apart possible (that would mean every dial turned exactly opposite), but simply with no agreement that counts. And the chance that two random codes visibly resemble each other falls exponentially with the number of dials; at thousands of dials it is, in practice, zero.

The consequence: the sky has room for an enormous number of stars that do not disturb one another. Anything that receives coordinates chosen at random is, with the same practical certainty, distinct from anything else that has received some. Randomness is not a weakness we tolerate; it is the source of separation.

2.3 The probe: a single operation

The probe has a single operation, and it is the same one the pilot will also use at the base: *to shift a point by a code*. You take the point's coordinates and a second code; for each dial, you turn the point's dial by the angle of the corresponding dial in the code. The result is another point. That is all.

The jump is this operation applied to the probe's position. A *key* is a code: "dial 1: +100°; dial 2: +30°; dial 3: +300°; dial 4: +45°". The pilot sends the key, the probe shifts its position by it and is at another point in the sky. It was at (10°, 200°, 95°, 300°); now it is at (110°, 230°, 35°, 345°); the third dial went past 360 and started over: 95 + 300 = 395, that is, 35. Then the probe reports the new position to the base. With the inverse key (each angle with a minus sign) it returns exactly to where it left from.

Three properties of shifting, all visible in the example, and all used below:

- *Shifting preserves shape.* If two points were close to each other and you shift both by the same code, they stay exactly as close, because each dial turned by the same angle on both. A whole constellation, shifted, keeps its drawing.
- *Shifting can be undone.* You shift back by the inverse code and you have the point from before, with no loss. Any string of jumps can be undone step by step.
- *The point after has no relation to the point before.* A key turns each dial by a different angle, drawn at random; so the starting point and the arrival point are, to anyone looking, two random points. The way back exists only for whoever has the key.

Two keys one after the other are still a key: the angles add up, dial by dial. From this comes a consequence that is both convenient and a real limitation: *order does not matter*. Key A then key B takes the probe exactly where B then A takes it, because 100° + 30° is the same thing as 30° + 100°. The probe's position remembers *which* keys have been used since the origin, not *in what order*. We come back to this in chapter 8.

Keep in mind: shifting a point by a code is called, in technical language, *binding* (bind). The jump is shifting the position by a key. The next piece is the same operation, used by the pilot on stars.

2.4 The word's star

Every word has, at the base, a *star of its own*: coordinates drawn at random once, when the word first appears, and never changed, never learned. "Ion" has its star, say (50°, 50°, 50°, 50°) in our four-dial example; "book" has another, (300°, 120°, 40°, 210°). They have nothing in common and never will. All the words' stars sit in a *catalogue*, at the base.

The star says nothing about what the word means; it only makes it recognizable wherever it is shifted, because the shape of a shifted star is preserved and shifting can be undone. Here there is already a difference from anything neural. A language model learns its word vectors slowly, from billions of examples, and an unseen word is broken into pieces and receives a vector guessed from its pieces. Here, an unseen word receives a new star on the spot, as good as any other. Nothing in the system depends on which coordinates a word received.

2.5 The sentence's chart: how a constellation is drawn

The probe has just reported the position (110°, 230°, 35°, 345°), and the word read is "Ion". The pilot takes Ion's star from the catalogue and *shifts* it by the reported position: the same operation as in the jump, only now the thing shifted is the word, and the code it is shifted by is the probe's position. Ion, (50°, 50°, 50°, 50°), becomes (160°, 280°, 85°, 35°) (the last dial: 50 + 345 = 395, that is, 35). This is a new star, "Ion shifted by the probe's position", and the pilot draws it on the chart. It no longer looks anything like Ion's star in the catalogue; but anyone who knows the probe's position can shift back and obtain (50°, 50°, 50°, 50°) exactly. This is how we say, in this story, "Ion was found where the probe was".

The chart is not a list of stars drawn side by side. It is a single code, in which the stars are *fused*: for each dial, you take the angle on the chart and the angle of the new star, draw them as two arrows on a circle, put them end to end and keep the resulting arrow, length included. You do this dial by dial. The chart therefore holds, on each dial, an arrow that may be longer or shorter; when you compare two charts or look for a star in them, what counts is the directions, the agreement dial by dial, not the lengths. The resulting chart partly resembles each star fused into it and nothing else. We call what results the sentence's *constellation*, and the chart, the document that holds it, is a *hologram*: all the stars are in it at once, superimposed. You can ask it questions ("do you contain Ion's star shifted by such-and-such a position?") and it answers a clear yes or no, but it cannot *list* its stars for you; from the sum of the arrows you cannot recover the arrows, you can only test candidates.

Two things about the chart must be kept in mind from the start.

First: a drawn star stays shifted by the position *of that moment*. The probe can jump on anywhere; Ion's star on the chart no longer moves. Each word carries the probe's position from the moment it was read, not the position at the end of the sentence.

Second: the chart has a capacity. The more stars you fuse, the harder each one is to recognize, because their arrows pull in different directions. With a few thousand dials, dozens of stars sit cleanly in a constellation; hundreds begin to blend together; beyond that, the chart slides toward a middling direction that resembles nothing. It is the only hard limit in the story, and the reason why long texts will be several charts linked together, not a single one.

2.6 The legend and the domains

Now we can say concretely what the legend is, on the example of the lending ledger. The world asks three questions: who gives (the agent), what (the object), to whom (the recipient). Whoever builds the world draws at random, once, one star for each question: *the agent's star*, *the object's star*, *the recipient's star*. Three random points in the sky, fixed forever, with no relation between them.

A named star defines a *domain* of the sky. The agent's domain is the set of all the stars you obtain by shifting a word's star by the coordinates of the agent's star: Ion's star shifted by the agent's star, Petre's star shifted by the agent's star, and so on, one for each word in the catalogue. It is, if you like, a whole copy of the catalogue, shifted to another part of the sky, with its shape preserved. The object's domain is another copy, shifted by the object's star; the recipient's domain, a third. Because the named stars are random, the three domains have nothing in common and are not confused, with the same practical certainty as in 2.2, although each contains all the words.

"Ion is in the agent's domain" therefore means exactly: on the chart is drawn Ion's star shifted by the agent's star. And for that to be so, the probe must have been *at the agent's star* when "Ion" was read: shifting by the probe's position (2.5) and shifting by the agent's star are then the same thing. To check who is in the agent's domain, you shift the chart back by the agent's star and look at which star from the catalogue has reappeared.

The same goes for the warehouse, only with other named stars: the supplier's star, the goods' star, the quantity star, the place star, the time star. "Supplier Alfa delivered 40 boxes to aisle B on Tuesday" should end up on the chart with "Alfa" in the supplier's domain, "boxes" in the goods' domain, "40" in the quantity domain, "aisle B" in the place domain and "Tuesday" in the time domain. If the sentence had not said "Tuesday", the time domain would have stayed empty on the chart, and that can be seen, as chapter 7 shows.

The legend is tiny, sits at the base, and is the only thing the world and the pilot have in common. The world uses it to draw the correct constellation of a sentence ("Ion in the agent's domain, book in the object's domain, Ana in the recipient's domain"). The pilot uses it to read his own chart. Nothing else is shared: the pilot does not know the world's facts, and the world does not know how the probe jumps.

2.7 The scanner: how the chart is read

You want to know who is in the agent's domain. The pilot shifts the whole chart back by the agent's star. What had been in the agent's domain returns to its star in the catalogue, recognizable. What had been in other domains is now shifted by a code that was not its

own and ends up at an unrelated point, that is, noise. The result is not a clean answer but a noisy code: the star of “Ion”, plus noise from “book” and “Ana”.

This code goes into the *scanner*. The scanner compares it with every star in the catalogue and reports the closest one, together with two numbers: how close it is, and by how much it is ahead of the next. Because the words’ stars are random and unrelated to one another, almost always a single one stands out, and the rest sit at noise level: “Ion”. When none stands out, or two stand out equally, that is information in itself, which we will use.

(In the four-dial example the scanner would not work: four dials are not enough for the noise to cancel out. With thousands of dials, the error rate falls exponentially with their number and depends on how many stars are fused in the chart and on the size of the catalogue; chapter 8 says where the limit appears. That is why there are thousands.)

With that, the story is described: a sky in which a point is just coordinates, a probe that jumps and reports, and a pilot with a catalogue of the words’ stars, a chart on which he draws the sentence’s constellation, a legend with the world’s named stars, and a scanner. Everything that follows is built from these alone.

3. Reading a sentence

3.1 The step

The pilot’s route book has one key for each *kind* of word (person’s name, verb, preposition, article…). These are the keys the pilot has learned, and the only things he has learned. The legend is not in the route book; it is used only when reading the chart, at the end.

At each word, the pilot does the same thing, identically:

1. He looks at what kind of word it is, takes from the route book the key for that kind and sends it to the probe. The probe jumps and reports its position. The position is, at every moment, the sum of all the keys sent since the beginning of the sentence.
2. He takes the word’s star from the catalogue, shifts it by the reported position and fuses it into the sentence’s chart.

That is all. The chart at the end is the sentence’s *state*: its constellation. It keeps no list of the words, nor their order, nor any grammatical tree. Only the words’ stars, each shifted by the position the probe was at when the word was read.

The whole learned part is the route book: one key per kind of word, a few hundred in all. The words’ stars, that is, the bulk of the vocabulary, are never touched. And the goal of learning, which we detail in chapter 6, is one alone: that the jumps dictated by the route book bring the probe, at the moment a word is read, exactly to the named star the legend requires for that word, that is, that they place it in its domain.

3.2 What it means for two sentences to “end up in the same place”

Take “Ion gives Ana a book”. The pilot sends the key for a name, then the key for a verb, then again the key for a name, then for an article, then for a noun. Along the way he drew

“Ion” shifted by the position after the first jump, “Ana” by the one after the third, “book” by the one after the fifth. Each star on the chart carries the point the probe was at in that moment.

Now “Ana receives a book from Ion”. Other words, other keys, another route. But if the route book is well learned, “Ion” is drawn after a string of jumps (name, receives, a, book, from, name) that brings the probe exactly to *the agent’s star*, that is, to the same point it was at when it read “Ion” in the first sentence. “Ana” ends up in the recipient’s domain, “book” in the object’s domain.

Two sentences mean the same thing when they draw the same constellation: the same words’ stars, in the same domains. The route does not matter. Three things must be kept apart here. The charts, as drawings, are not identical: function words and different routes leave different traces, and the charts are only *similar*; that similarity is what training uses. The frames read from them, domain by domain, are *equal*: the same stars in the same domains. “The same meaning” means the second thing, and that is how it is measured: you compare the two charts through the legend, domain by domain.

3.3 Reading the chart: the frame

You want to know who the agent is. The pilot shifts the chart back by the agent’s star and gives the result to the scanner: “Ion”. He does this for every named star in the legend and has the *frame*: agent Ion, object book, recipient Ana. The frame is not computed from the chart by some other program; it *is* the chart, read through the legend.

With each answer come the scanner’s two numbers: how well the winner matches, and how far behind the second is. The difference is the *margin*. Large margin: a single word was found in that domain. Small margin: two were found, or one was drawn from a point lying between two named stars, so it is half in two domains. The margin is where “I’m not sure” lives.

3.4 Sentences within sentences: the legend is relative

“Maria says that Ion gives Ana a book.” Our ledger also has in its legend questions for reports: *who says?* (the speaker) and *what is said?*. But what Maria says is not a word, it is a whole sentence, with its own agent, object and recipient. If the pilot drew “Ion” in the agent’s domain, we would no longer know whether Ion is the agent of the main sentence or of the reported one. We need a separate set of domains for the inner sentence, which nevertheless has *the same names*.

The solution is again a star. The legend has, besides the speaker’s star, a star for the role “what is said”: the *“reported”* star. And here it becomes visible that the named stars are not, in fact, absolute points, but *displacements from the sentence’s origin*: “the agent’s star is at +the agent’s key from the origin”. For the reported sentence, the origin is different: it is the point the probe reaches by jumping with the “reported” star. The agent’s star *of the reported sentence* is “origin + reported + agent”: a new point, unrelated to the agent’s star of the main sentence, and yet with the same name in the legend, read from another origin. The

whole constellation of the legend shifts at once, with its shape preserved, because shifting preserves shape.

The pilot has learned that the kind of word “that” calls for exactly the “reported” jump. From there, he reads the inner sentence with the same keys from the route book as usual. “Ion” is drawn in the agent’s domain from the reported origin. Maria has remained in the speaker’s domain, relative to the main origin.

To read the inner agent, the pilot undoes the path: he shifts the chart back by the “reported” star (returning to the origin of the main sentence), then by the agent’s star, then the scanner: “Ion”. Two sentences one inside the other (“Maria says that Dan believes that Ion…”): he undoes two “reported” jumps. The depth of nesting is, literally, the number of origins shifted; we call it the *level*.

And from here comes something we have measured: because “shift the origin” is the same key every time, a pilot trained on sentences with at most two levels reads a sentence with three correctly, with no new learning. It is the same jump, made once more. Trained up to two levels, he read three without error, while a neural network trained on the same sentences, with the same input, guessed at random at level three. Nothing special was built for this: each level is the whole legend, read from another origin, and origins can be shifted any number of times, because a key can be used any number of times and undone just as many.

3.5 The beacons on the chart: how a word knows which domain to go to

The step in 3.1, in which a word’s key depends only on its kind, has a weakness you see immediately. “Ion gives…” and “Ion receives…” begin with the same word, of the same kind, hence with the same jump. But Ion is the agent in the first and the recipient in the second. When the pilot draws Ion, after the first jump, he has no way of knowing which domain it must reach; what follows has not been read yet.

What happens in practice: the key learned for names settles on a compromise, a point halfway between the agent’s star and the recipient’s star. The sentence is still read correctly, because when you ask “who is the agent?”, Ion is *roughly half* there (how much exactly depends on how closeness is measured), and nobody else is there at all. But the margin is thin, and with a few such compromises in one sentence the reading becomes muddy.

The full version of the pilot solves this with two extra habits, still without knowing anything about the world.

First: some words put beacons on the chart. We take the verb as the example because it is the easiest to follow; which words actually put beacons is learned, and the result is stated a few paragraphs below. When the pilot reaches “gives”, the word announces: “after me come an object and a recipient”. The pilot notes two *beacons* on the chart: “the object will be at the position reported now + the relative key for object”, “the recipient at the current position + the relative key for recipient”. The relative keys are also learned, per kind of word, and sit in the same route book. A word that arrives later checks whether there is a

beacon for its kind; if so, its star is shifted by *the beacon's position*, not by the position reported by the probe. Beacons are marks on the chart, not lights lit in the sky, and they are erased at the end of the sentence.

Second: a word that arrives too early is held aside. "Ion" at the beginning of the sentence does not yet have any beacon waiting for it. The pilot does not draw it; he holds it aside, and moves on. When he reaches "gives" and the beacon "an agent follows" appears, Ion from aside is the first to match it, and is drawn then, shifted by the beacon's position, that is, in its proper domain. This is how the first word receives its domain, although its role becomes visible only later.

Which kinds of words are allowed to put beacons is not for us to decide. Each kind of word has, in the route book, besides its key, a *gate*: a number between zero (never puts one) and one (always puts one), learned from the same objective as everything else. We would have set verbs and prepositions to one. The pilot chose otherwise, on his own: the words that mark a relation ("to", "from", "of", "that", a comma) have their gates open; names have them closed; verbs, surprisingly, closed too. The explanation, once seen, is simple: if the words held aside can be placed by what follows, the preposition already says everything the verb would say. The same choice reappeared, without our asking for it, when we trained the pilot on real questions typed by people about tables. The structure of "who opens a frame" is, in this system, a result, not an input.

3.6 Three drawings for the same word

We now have two ways of finding out which position a word's star is shifted by: the beacon's position (precise, but fragile: if a construction is new and a beacon is missing, the word stays aside or is drawn wrongly) and the position reported by the probe, that is, the sum of the generic keys of the preceding words (robust, but unclear, as we saw with Ion). There is also a third, cruder one: a position that depends only on *the kind of the preceding word* ("after a preposition, a name"), almost never wrong and almost never precise.

Because fusing into the chart is cheap, the pilot does not choose between them. He draws the word's star *three times*, shifted by the three positions, and fuses all three copies into the same chart. When the sentence has a familiar shape, the three positions are the same point and the drawing is sharp. When the sentence uses a never-before-seen construction, one of the positions is off, but the other two still bring the star close enough to its domain for the scanner to recognize it. A wrongly learned key no longer destroys the reading; it only blurs it. On new constructions, this redundancy raised correct reading from six in ten to nine in ten.

With this, the pilot is complete: a route book with one key per kind of word, relative keys for beacons, one gate per kind, and the step from 3.1, repeated identically at each word, with three drawings. Everything that follows is about how the route book comes to be good, and what you can do with the chart at the end.

4. Why two sentences of different lengths can mean the same thing

“Ion gives Ana a book” has five words. “Ana receives a book from Ion” has six. “The book was given to Ana by Ion” has eight. A reader who counts words, or who remembers their order, sees three different things. The pilot sees one. This chapter says why, step by step, because this is where it is clearest what the chart is and what it is not.

Length is a property of the route, not of the chart

The probe makes one jump per word: five jumps for the first sentence, six for the second. The routes have different lengths. But the chart does not record the route; it records *where the probe was when each word was read*. On the chart there is no “third word” or “after six jumps”. There is Ion’s star shifted by the agent’s star, and that is all. If two routes of different lengths bring the probe to the agent’s star at the moment “Ion” is read, the two charts have exactly the same star in the same place, and nothing on the chart says any longer how long the route to it was.

This is not an accident, it is a choice. None of the three positions the pilot uses to draw a word (3.6) contains the word’s ordinal number in the sentence: the probe’s position is the sum of the preceding keys, the beacon is a displacement from the word that opened the frame, and the third depends only on the kind of the preceding word. At one point we also tried a position that contained the ordinal number; it tied the reading to length and order, that is, to exactly what paraphrases change, and we set it aside (chapter 15 says what remains to be measured there).

Function words move the probe, but do not enter the frame

“To”, “a”, “from”, “by” are words too, and the pilot treats them like any other: each has a key, the probe jumps, the word’s star is drawn on the chart where it landed. But the points they land on are not named stars. When the pilot reads the chart through the legend, he asks only the named domains; the star of “from” sits at a point unrelated to any of them and answers no question. Function words are the scaffolding of the route: without them the probe would not get where it must, but once it is there, they no longer matter. A sentence with more function words has more scaffolding on the chart, at unnamed points, and the same frame.

That is also why we compare charts *domain by domain*, not as two drawings laid on top of each other. Two charts of the same event are not identical point by point, because the scaffolding differs; they are identical where it matters, in the answers to the legend’s questions.

Different words, the same destination

The hard question remains: why do “gives” and “receives … from” bring the probe to the same named stars, when they are different words, with different keys? Because that is how the route book was learned. In training, the two sentences received the same world’s constellation (6.1), and the pilot’s only freedom was the route book. To satisfy both, he had to find keys under which “name, gives, name” and “name, receives, …, from, name” bring the

names to the same stars. Beacons make this possible without contortions: in our example "gives" puts beacons for object and recipient relative to its own place, and "from" puts a beacon for the agent relative to its place. (In the learned route book, beacons are mostly put by relation words, "to", "from", not by the verb; where the verb is silent, the other two drawings of 3.6 bring the word to its place.) The word that follows takes the beacon, wherever it is in the sentence. The length between "from" and "Ion" does not matter, because a beacon is a destination, not a number of steps.

The different verbs, "gives" and "receives", also reach the chart, in the predicate's domain (a domain of the full legend; the first worlds measured did not have it, as chapter 13 says). Whether they count as the same event or as two different events is not for the pilot to decide but for the world's legend, which says which verbs describe the same event (6, "Which verb?"). The pilot only draws them consistently.

One example traced, and one that does not match

"Ion gives Ana a book": the probe leaves the origin; at "Ion" the pilot holds it aside (no frame open yet); "gives" opens beacons for agent, recipient and object; Ion takes the agent's beacon, "Ana" the recipient's, "book" the object's; "a" stays in the scaffolding. The chart: Ion in the agent's domain, book in the object's, Ana in the recipient's, "gives" in the predicate's.

"Ana receives a book from Ion": "Ana" held aside; "receives" (in our example) opens beacons; Ana takes the recipient's beacon (for "receives", the one who receives is the recipient, and that too is in the route book); "book" takes the object; "from" opens the agent's beacon; "Ion" takes it. The chart: Ion in the agent's domain, book in the object's, Ana in the recipient's, "receives" in the predicate's. Six words, the same frame, except for the predicate, which the legend declares the same event.

Now "Ana receives a book from Ion on Tuesday". Eight words, and one more star: "Tuesday" in the time domain. The two charts *no longer* say the same thing, and the difference is located exactly: the same answer to who, what, to whom, one more answer to when. It is the third kind of "near" from chapter 8, and it is all that "longer" means when length brings information: one more domain filled, not a bigger chart.

When length does matter, and what the chart does then

Two words instead of one in a domain ("Ion and Petre"): the domain is crowded, with two stars, and that is the correct meaning, not an error. A missing word ("Ion gives a book", with no recipient): a domain stays empty, and that again is the correct meaning, with the gap in plain sight. A repetition for emphasis ("very, very late") draws the same star twice in the same place, which makes it a little stronger and does not change the frame; if a world wants to distinguish "late" from "very late", it must have a domain for it. And the known limit: when the shape of the sentence is revealed only at the end ("…, says Maria"), the length before it has already been drawn relative to the wrong origin; a reader who moves forward cannot repair that, as chapter 8 says.

On constructed worlds, different formulations of the same event, with different numbers of words and different orders, reached the same frame, domain by domain; it is the result the

whole document started from. On real text, the signal comes from the questions about tables, where questions of different lengths led to the same column in eight cases out of ten.

5. The same word in several worlds: polysemy, fields of knowledge, ontologies

A word like “bank” means one thing in a text about finance and another in one about a river. “Energy” means one thing in physics and another in nutrition. “Field” means one thing in algebra, another in agriculture, another in physics. A reader with a single star for each word seems unable to cope. This chapter says how things actually stand, and what we do *not* claim.

A word about vocabulary first: in this book, *domain* means the region of a role (the agent’s domain, the quantity domain). For a field of knowledge or activity, finance, physics, a warehouse, we say *world*, because that is what we called it in chapter 1: the thing the sentences are about, with its list of questions.

The star is the word, not the sense

The star of “bank” is one, fixed, the same everywhere. That is not a weakness; it is a consequence of the definition in 1.1. The meaning of a sentence is “which questions it answers, and with what”. The meaning of a *word*, by the same definition, is “which questions it answers, in the world in which it is read”. In the world of finance, “bank” answers questions like “who granted the loan?”; in the world of geography, “where did the boat stop?”. The same star, different legends, different domains. The pilot never has to decide *what kind of bank* it is; it only has to place it in the right domain of the world it works in. What a bank is, in that world, the database knows, and that is its business.

Put differently: polysemy does not live in the stars, it lives in the legends. Two senses of the same word are two legends in which it answers different questions. That is also why the stars can stay random and fixed (chapter 8): they have nothing to learn about sense, because sense is elsewhere.

What the legend alone does not resolve is a double sense in the *same* world and the *same* domain: “at the bank”, as a place, may be the river bank or the bank’s building, and if the star and the domain are the same, the chart does not tell them apart. There, disambiguation remains a matter of context, of a sense identifier in the legend, or of the database. The legend narrows ambiguity; it does not remove it.

When the kind of the word is ambiguous

There is one ambiguity that concerns the pilot directly: when the same word can be of two *kinds*, with two different keys. “Run” is a noun or a verb; “book”, likewise. The kind decides the jump, so here the pilot does have to choose. The mechanism for this already exists, and it is the one from 3.5: a word whose kind is not yet clear is held aside, or drawn halfway between the two positions, until a beacon for one of the kinds takes it. “Book” after “a” takes the noun’s beacon; “book” after “please” takes the verb’s. If no beacon comes, the word stays drawn half in two domains, and the chart reports that like any crowded domain:

“I do not know whether this is a thing or an action”. It is the same thing we do with the first word of a sentence, whose role is not yet visible; lexical ambiguity asks for nothing new.

The world is a first-class object

The natural question: if “bank” means something different in two worlds, should not the *jump* the probe makes at “bank” also be different in each world? Yes, and so it is; it is worth saying plainly what belongs to the world and what does not.

The only thing all worlds share is the sky, with the words’ stars: the star of “bank” is one, because it is the identity of the word, not of the sense. Everything else belongs to the world. A world has its *legend* (which questions it asks, with their named stars); its *kinds* of words (in the world of finance “bank” is of the kind of institutions, in geography of the kind of places, and the kind decides the key); its *route book* (the keys are learned on that world’s sentences, so the jump at “bank” is different in each world, because the kind is different and the route book is different); its *background* for the remainder (what an understood sentence looks like there); and its *database*, where the facts live. The pilot keeps one route book for each world it works in. The world is, therefore, a first-class object in the story, not a vague context: it has a name, it has parts, it is trained separately and it can be changed without touching the other worlds. When in ordinary language we say “scientific field” or “training domain”, in the story we say *world*, and it is a citizen with full rights.

There is also a second way, which we have not built: the world itself as a star. Just as the “reported” key shifts the whole legend to another origin, a world’s star could shift the legend of finance to one part of the sky and that of geography to another, with a single pilot and a single route book. It would be more economical, but it would require the kinds of words to be shared across worlds, and that is exactly where polysemy lives; the superposition of kinds from the previous section would then be needed. It is a research direction with a clear trade-off: one route book per world is simple and isolates the worlds; a world star is economical and links them.

Several worlds at once

A system installed in an enterprise usually has several worlds: invoices, deliveries, leave requests. Each has its legend and its route book, as said. A new sentence is read by all the pilots at once; each costs microseconds. Then the charts are compared: with the pilot of the right world, the domains fill with a clear margin and the residual is small; with the others, the stars fall outside the legend and the residual is large. The world of a sentence is recognised by how well its legend fits, like a key in a lock. No separate “what is this text about” classifier is needed: the fit is the classifier, and it comes with the same numbers, margin and residual, as any other reading.

Two limits, stated openly. When two worlds share vocabulary and have similar questions, the fit is weaker, and how much weaker is measured, on constructed worlds with controlled overlap. And when a single sentence mixes worlds, as literature, metaphor and jokes do, the pilot has nothing to fit: it reports a large residual in every legend, that is, “this belongs to no world I know”. That is not a promise we make; literature is not the target.

Do we discover ontologies?

No, and it matters not to give that impression. An ontology, in the sense used in computing, is a list of kinds of things and of relations between them, with names. A world's legend *is* a small ontology, a few dozen lines, and it is written by people, not discovered. The kinds of words come from a statistical grouping of the corpus, which we checked: they coincide with what a classical method gives, and we do not claim them. What truly emerges from training is one thing: which kinds of words open frames (the gates), and that is less than an ontology.

What we do, instead, is *populate* a given ontology: reading a whole corpus produces a list of frames, that is, facts with roles, that is, a knowledge graph relative to the legend: who delivered what, to whom, when, how many times. All the stars that ever reached the supplier's domain are, taken together, that world's vocabulary for "supplier"; nobody wrote it, it resulted from reading. It is an inventory, not a theory, but an inventory verifiable sentence by sentence.

And there is one more thing, which belongs to ontology without being its discovery: the chart can signal that *a question is missing*. If a world's sentences systematically say something the legend does not ask about (every delivery has a time, but the legend has no "when"), those stars always fall outside the legend, and they fall the same way: the residual is not noise, it has a direction. The signal is "something is being said here that I do not ask about", and the answer is a new question in the legend, added by a person and then learned by the pilot with its route book. This is the loop of 9.4, "a boundary that learns", stated concretely: the ontology is not discovered by the system, it is *maintained* by people with the system's signal.

The same words, different theories

There remains the most interesting case in the question we started from: the same word in two theories, "mass" in Newton and in relativity, "energy" in thermodynamics and in nutrition. In our terms, two theories are two legends: the same words answer different questions and stand in different relations. A text read with both legends gives two charts, and the difference between them, domain by domain, says exactly where the two theories use the same word differently. It is a direction, not a tool: it requires legends written for theories, which nobody has done yet, and texts to fill them. But it is the same mechanism as the difference between two versions of a regulation in 11, applied to something harder.

6. How the pilot learns

6.1 What he receives: the world's constellation

Nobody tells the pilot which sentences are paraphrases. But it is not "without any supervision" either, and this must be said plainly. Each training sentence comes with a *world's constellation*: the world, which knows the event and has the legend, draws the correct chart itself: "Ion's star shifted by the agent's star, the book's by the object's star, Ana's by the recipient's star, fused". In some of the worlds we built, the world's constellation describes the event; in others, its consequence ("Ana has the book"). In all of

them, the legend (the roles) is fixed in advance by whoever built the world; what the pilot learns is *how to get* the probe to its named stars, word by word.

Training is then a single game, repeated: the pilot reads the sentence with his route book, draws his chart, and his chart is compared, dial by dial, with the world's constellation. The better they match, the better; the keys in the route book are adjusted slightly, in the direction that would have brought the two charts closer. That is all. Supervision is therefore per sentence and structural (a constellation with roles given in advance), not a list of which sentences mean the same thing.

6.2 Why this forces meaning

Two sentences about the same event receive the same world's constellation. So they are obliged to draw the same chart. The pilot's only freedom is the route book: one key per kind of word, a few hundred keys in all. With a few hundred keys, memorising sentences one by one is practically excluded; what it can find are regularities, and that is exactly what we want. So the only way he can satisfy both demands is to find keys under which both routes bring the probe to the same named stars. Equivalence is not labelled on pairs; it is induced by the shared targets, and it is the solution of a constraint.

We tried the obvious alternatives and they failed instructively. If you ask the pilot to predict *the next word*, like a language model, all routes become alike and the domains disappear: nothing forces him to separate the roles. If you give him a memory that *looks up* the world's constellation starting from his chart, he predicts perfectly and aligns nothing: a lookup can take three different charts to the same answer without ever making them the same chart. Only when the pilot must *produce* the world's constellation through a fixed, rigid operation (his chart, compared directly) is he forced to make the charts coincide. Rigidity is the essence.

6.3 What is learned, and how little

A few hundred keys (one per kind of word), a few hundred relative keys for beacons, and one gate per kind. All of it is on the order of a megabyte at the sizes we use, a few megabytes at larger sizes. The words' stars, the bulk of the vocabulary, are random and fixed. The named stars in the legend are random and fixed. The whole trainable part is the route book.

That is why training takes minutes, why a new word costs nothing, and why the pilot knows nothing about the world: he has nowhere to keep facts. Facts live in the world, which in an installed system is a database or a reasoning engine, where they can be inspected and corrected. The pilot holds only how the language places words into domains.

7. What remains on the chart after a sentence, and what it is good for

After a sentence has been read, a single chart remains: its constellation. On it, recoverable but not listed, are the word stars, each shifted by the position in which it was found. Next to it is the legend: the named stars. And between the chart and the legend there are distances that can be read *without understanding anything*: how much of the constellation is

explained by domains, how cleanly each domain is occupied, whether any domain the world wants filled looks empty.

This is the property on which everything else rests. The chart is not measured against itself; it is measured against the legend. So the pilot can say, without understanding a sentence, *that* it has not understood it, and in what way:

- *Nothing recognised in a domain.* You shift the chart back by the recipient's star and no star in the catalogue stands out. Either the sentence did not say who the recipient is, or it said so with an unknown word; to tell the two apart, a small extra device is needed, described in chapter 8. Either way: *recipient unrecognised*.
- *A crowded domain, or one half occupied.* Two candidates, a small margin. Ambiguity, and the pilot can name them both.
- *Stars left unexplained.* You take each recognised role, reconstruct the constellation these recognitions would produce, and subtract it from the real chart. What remains are stars the legend could not place in any domain: words read, jumps made, drawings placed, but not in named domains. The sentence is about something outside the world, or it is incoherent, or it uses a construction for which the route book has no keys. Measurable as the size of that remainder.
- *A key that could not be used.* A word without a kind (any word receives a new star; only words with a kind receive a key); the route breaks there, and every drawing after it carries a position reached with a guessed key. Visible as a collapse of the margin for all the roles after that word.

Two cautions, because they are easily forgotten. A clean reading is not a correct reading: a word can be drawn confidently in the wrong domain, and how often that happens must be measured, not assumed. And "unrecognised" is a statement about the pilot's legend and catalogue, not about the sentence.

A language model has no legend. Its state is a point with nothing to measure it against, and it answers with the same fluency whether its route passed through named domains or through emptiness. Ours reports where the route left the legend. Every application below is this one property, in a different setting.

8. Phenomena you can follow on the chart

Everything in this chapter happens inside the story described and can be measured there. None of it is a rule written by us. We go slowly, because these are the phenomena that build intuition, and because each is also a diagnostic that can be shown to a user.

The empty domain, and why "empty" is harder than it looks

You shift the chart back by the recipient's star and compare the result with every star in the catalogue. Either one stands out with a clear margin (*recipient: Ana*), or none does.

Here is a trap it is easy to fall into. You might hope to tell "nobody was found in this domain" from "someone was found whom I do not know" by how *much light* comes back. You cannot, and it is worth saying why. Shifting is rigid: it shifts the whole chart, every star

in every domain, at once. What comes back always has the "weight" of the entire constellation, with the stars of the other domains scattered as noise. The amount tells you nothing; only recognition does. "Ion gives a book" and "Ion gives Zxyq a book" look the same from the recipient's domain: no known star stands out.

To tell them apart, a small, explicit device is needed, standard in this family of systems: together with every word star it draws, the pilot also fuses a fixed *marker*, always the same, which means "something was drawn here". Then the recipient's domain can be asked two separate things: is the marker here (something was drawn), and which star is here (and do I know it). "Ion gives a book": no marker, nothing drawn, *recipient not given*. "Ion gives Zxyq a book": marker present, a star not in the world's catalogue, *recipient present, unknown*. The device is cheap; without it, there is a single reading: *recipient unrecognised*.

The crowded domain

Sometimes two stars come back from the same domain with similar strength. There are two ways this happens, and they look the same from the outside, but not from the inside.

One: the sentence really did put two things there. "Ion and Petre give Ana a book." The agent's domain has two stars fused on the chart, and both come back at roughly half strength. It is not confusion; it is a set, correctly drawn, and the pilot must report both. The chart alone does not, however, distinguish "Ion and Petre" from "Ion or Petre, we do not know which": both would be two stars in the same domain. The distinction requires a mark in the legend, one for "and", another for "one of two", just as the occupancy marker distinguishes empty from unknown.

Two: a single word was drawn from a point lying *between* two named stars. This is the fate of the first word with the simple pilot of 3.1: "Ion" is read before the verb, nobody yet knows whether he gives or receives, and the probe's position is halfway between the agent's star and the recipient's. Ask the agent's domain and Ion comes back at half strength; ask the recipient's domain and again at half. The reading works, because nobody else is in either of the two, but the margin is thin.

Beacons repair the second case and leave the first alone. When the word that sets beacons arrives, Ion, held aside, is drawn from the position of the appropriate beacon and comes back at full strength. Before-and-after shows up as a jump in margin; in our measurements, about threefold. The crowded domain of the first kind stays crowded, because it must.

Stars outside the legend, or how the pilot knows a sentence makes no sense to it

You read every domain, take the recognised stars, and reconstruct the constellation they would produce: each star shifted by the named star of its domain, all fused. You subtract the reconstructed constellation from the real chart. What remains is the part of the sentence the legend could not explain. We call it the *remainder*, and it deserves an explanation without mystery, because most of the diagnostics rest on it.

The remainder does not appear at reading time. Reading a domain, shifting the chart back and running the scanner, is an exact operation: shifting is an isometry, everything moves intact, nothing is "left behind". That is also why you cannot tell from *how much* comes back

whether the domain is occupied; the whole chart always comes back, shifted. The remainder appears at *comparison*: once you have read every domain, you have the frame (agent Ion, object book, recipient Ana); you draw again, from scratch, the chart that exactly these answers would produce; and you subtract it from the real chart. It is a division, in the statistical sense: chart = (what the legend explains) + (remainder). The frame is the quotient, the remainder is the remainder. The quotient says what was understood; the remainder says how much of the sentence could not be placed in any named domain.

Two clarifications make it honest. First, the remainder is never zero: function words drawn at unnamed positions, the extra copies from the three drawings and the noise of superposition leave a *background* that sits in no domain; the background is measured once, per world, on understood sentences, and what counts is *how much* the remainder exceeds it. Second, the remainder is computed after the reconstructed constellation is brought to the chart's scale: charts are compared by direction, not by length, so a chart three times "longer" is not three times less understood. With these clarifications: for an understood sentence the remainder stays at the background; for one not understood it rises, and where it comes from depends on the reason:

- *Known words, in an order never travelled.* Each word used its key, but the sequence of jumps took the probe to points the legend does not name. The stars are real, they would be recognised if you knew what to shift the chart back by, but no named star is the right one. This is the shape of a new construction: the pieces are familiar, the route is not.
- *Unknown words.* Each is a new, random star, shifted by wherever the probe happened to be. Random stars at arbitrary points: they add weight to the chart and contribute nothing to any domain. This is the shape of nonsense, or of a foreign vocabulary.
- *A sentence about something else.* The words are known, the construction ordinary, and the legend simply has no domains for this kind of event, because the world the pilot was trained on does not contain it. The stars land at points perfectly well formed and perfectly nameless. This is the shape of a question from outside the field.

Notice what this measurement is and is not. It is not "the sentence makes no sense". It is "my keys do not take this sentence into my legend". A sentence can be full of meaning for a person and still fall outside the legend of a pilot trained on invoices. The pilot knows the limits of its own legend, and precisely this kind of ignorance is useful.

The broken key

An unknown word is not just unrecognised; it has no key either, because it has no kind. The pilot, on reaching it, sends a guessed key, and everything it reads afterwards is drawn from positions that carry that wrong key. The drawings before the unknown word are good; those after it are displaced by an unknown amount. Ask for the roles and you see the pattern: those filled before the break come back clean, those after it, at noise level. The break can be located: the pilot can say *I lost the thread at word six*. The three drawings

soften this, because they do not all break at once; but something after the break is always blurred.

Noise, and why the chart has a capacity

Every fused star adds a little noise to every domain. When you shift the chart back by the agent's star, you also bring back, faintly, everything that was drawn in the other domains, shifted by a code that was not its own and therefore scattered at random. With few stars, the scatter is well below the signal. With many, it accumulates, and at some point the word in the agent's domain is no longer clearly above the scatter from everything else.

How many "many" is depends on how many dials the coordinates have. With a few thousand, tens of stars are clean, a hundred or two still readable with care, and beyond that the chart slides towards the middle, where nothing is recognisable any more. It is not a defect; it is the geometry. This is why a document is not one chart but several charts linked by keys, each holding what one sentence or one event drew. The "reported" jump is already such a link: the inner sentence is its own constellation, shifted whole to another origin, within the chart of the outer one.

Negation as a star, and where its scope lives

"Ion does not give Ana the book." The legend provides a star for negation. The question is which positions carry it. If all the drawings of the event carry the negation star, the negation belongs to the sentence: it is not the case that Ion gives. If only the drawing of "Ion" carries it, before the agent's star, the negation belongs to the agent: it is not Ion who gives. If only Ana's drawing carries it, the negation belongs to the recipient: it is not Ana who receives, and who does receive remains an open question (not "nobody": "nobody" is a word of its own, with its own drawing, which the legend treats separately).

The scope of a negation is therefore not a label; it is *which words were drawn from positions that contain the negation star*. You read the chart back and see: shift the whole chart by the negation and then read the domains; if all the stars reappear in their places, the negation belonged to the whole sentence; if only Ion reappears, it belonged to him; if nothing reappears, the negation was not there. The pilot reads which of the cases it is, for eight ways of saying the negation, without a rule for any of them.

And here is why a bag of words cannot do this. For the bag, "not" is a word like any other: the negated sentence and the affirmed one share all their words but one and are nearly identical. In our story, the negated sentence has its stars shifted by one extra key, and the two constellations are far apart, almost like two random points. The difference between "the same words" and "the same meaning" is the difference between counting the stars and asking where they were drawn.

Ambiguity held open

"Every child reads a book." One book for all, or one each? The sentence does not say. A reader forced to choose will choose, and will be wrong half the time without knowing it.

In our story, the two readings are two different arrangements of the quantifier domains. We trained the pilot only on unambiguous sentences, each with a clear arrangement, and then showed it ambiguous ones. What we observed is that its chart falls between the two arrangements: ask for either reading and it comes back, at nearly equal strength. We did not teach it to hesitate, and nothing in the geometry guarantees this result in advance; it is an experimental finding, and a welcome one. The pilot reports both readings, and a later word that fits only one ("the same book") would close the other. One clarification: the superposition of two readings does not also remember the link between the choices (which noun goes with which reading); it is a representation of doubt, not of the two whole meanings. It is the same phenomenon as the first word drawn at half strength, in a different setting: a star drawn between two domains until something settles it.

The level never seen, and the key that does not exist

Why can the pilot read a sentence with three levels when it was trained on two? Because "shift the origin" is a key: twice is a key, three times is a key, and undoing it is the same number of steps back. The third level has nothing new; it is the same jump, once more.

And why did it fail, the first time, to read a negation *inside* a reported sentence? Because that combination required a chaining of keys the pilot had never performed: negation-at-level. Not a new key, but a new chaining, for which it had no learned relative key. Once that chaining had appeared in training at one level, it read it at the next level without further training. The pattern deserves to be stated for what it is: the pilot composes what it has keys for. A chaining it has never performed is not blurred, it is missing. This is what "generalises one level beyond what it was trained on, once the chaining has been seen one level down" means, and it is also exactly where the method fails: on a combination of known operators it has never seen together.

A new construction: a missing beacon

"The book, Ion gives it to Ana." Known words, an order never travelled. No key is wrong in itself: every word has its kind and the kind's key. What breaks is something else, and it is worth saying exactly what, for each of the three drawings. *The beacon*: the new construction opens the expectations in another order; "the book" arrives before any beacon and may be taken by an unsuitable one or by none; the drawing by beacons is the most fragile here. *The probe's position*: the sum of the keys is a new combination of known keys, hence a point the probe has never reached; not nonsense, but not exactly at the named star either; the drawing is displaced. *The kind of the previous word*: "after an article, a noun" is the same as in any sentence; the drawing is crude, but untouched.

With a single drawing, the one by beacons, the word would be lost. With three, the other two bring it close enough to its domain for the scanner to recognise it, with a smaller margin. The reading is blurred, not destroyed. That is the entire content of the result about redundancy: a new construction costs part of the margin instead of all of it.

Structure announced afterwards

“Ion gives Ana a book, says Maria.” By the time “says Maria” arrives, the first sentence has already been drawn on the chart, relative to the main origin, as a plain assertion. The “reported” jump comes too late: the stars are already drawn, and a pilot that only moves forward has no way to lift them and shift them to another origin after the fact. The same goes for a comma that reorders a sentence, and for any construction whose shape is revealed only at the end.

It is a real limit of forward reading, and we declare it as such. It is also, precisely, what “holding aside” does for single words, extended to whole sentences; the only place where the idea that a word may need to be re-placed by what follows survives, in the form the data forces on it.

Order, and what commuting keys cannot tell apart

The keys commute. “Reported” then negation takes the probe exactly where negation then “reported” takes it. So the inner sentence in “Maria says that it is not the case that P” and the one in “it is not the case that Maria says that P” have the same origin. The two sentences can still be told apart, but only because *something else* differs: in the second, Maria’s drawing also carries the negation star. Where nothing else differs, the order of two operators is lost.

This is the deepest structural limit of the story as we use it, and it must be declared as such: a position remembers which keys were used, as a multiset (repeating the same key counts, order does not), not as a sequence; the paths through the sky form a grid, not a tree. It is the same fact, seen from another side, that gives the pilot the freedom to compose (“the same jump once more” requires no learning). In the variants we have weighed, keys that do not commute, through permutations, or a separate “reported” star for each level, order is bought with part of that freedom. It is not a mathematical impossibility (there are operations that preserve order and compose), but a trade-off of the concrete variants, and it is measured: we measured what the commuting version can and cannot do, and sentences in which order alone carries the meaning are tested with both remedies.

Which verb? The predicate is a domain too

“Ion gives Ana a book”, “Ion lends Ana a book”, “Ion sells Ana a book”: the same participants, the same domains, three different events. If the chart holds only who was found in the agent’s, the object’s and the recipient’s domains, it cannot tell them apart. The verb itself must be drawn in a domain of its own, the predicate’s domain, and the legend must have its star. In our early worlds it did not: giving and selling had received the same world’s constellation and were therefore the same event for the pilot. That was a gap in the way we built those worlds, not a property of the story, and it is being closed. It also answers a natural question: “gives” and “receives” arrive at the same constellation because the world says they describe the same event; “lends” arrives at a different one because the world says so. The world decides what counts as the same event; the pilot only has to be consistent with it.

Several events, and the same name twice

One chart, one event. If two events are fused into a single chart with nothing to keep them separate, the participants mix: the agent of the first and the agent of the second are both in the agent's domain, and nobody can say any more who did what to whom. The story has the obvious remedy, each event shifted to its own origin before fusing, exactly as a reported sentence is kept separate from the one reporting it; but we have used it only for reporting, not for "Ion gives Ana a book and Petre lends her a pen". Until it becomes general, the honest range of a chart is a single event. The same goes for a name that appears twice in different roles: they are two drawings in two domains, they can be told apart, but the margin drops a little, as it drops whenever the same star is drawn in two places.

Three kinds of "close"

Because the chart keeps both the word stars and the domains, two sentences can be close to each other in three ways, and the chart tells them apart.

- The same stars in the same domains: the sentences mean the same thing. "Ion gives Ana a book" and "Ana receives a book from Ion".
- The same stars in different domains: the sentences are about the same things and mean something else. "Ion gives Ana a book" and "Ana gives Ion a book". A bag of words sees no difference at all; here the two constellations are far apart.
- Different stars in the same domains: the same shape, different content. "Ion gives Ana a book" and "Petre lends Maria a car". The same frame, different fillers; structurally equivalent.

A similarity score crushes the three into a single number. Our chart keeps them as three answers, and a comparison of two texts can say what kind of closeness it found and in which domains.

What the pilot does not remember

Between sentences, nothing. While it reads a sentence, the pilot has a small working set, the beacons on the chart and the words held aside, but it discards it at the end; what survives is only the chart. The next sentence starts a new one, and the probe sets out again from the origin. It is a deliberate emptiness: the pilot has no memory in which to accumulate beliefs, and so it cannot drift or confabulate over the course of a conversation. Memory is external: the frames read are stored in a database or a reasoning engine, and a later sentence that refers back ("she reads it") is resolved by linking its chart to the stored frames, through a search, not through a memory inside the pilot. That linking is the precondition for anything at the scale of a document. The next section says how it is done.

The pronoun: a borrowed star

"Ion gives Ana a book. She reads it." For the pilot, "she" is a word like any other: it has its star in the catalogue, it is of the kind "feminine pronoun", and it lands on the chart of the second sentence in the agent's domain. The chart is correct and, at the same time, does not say who the agent is: it says *she*. Nothing in the sky can say more, because the star of "she"

has no connection to Ana's star; the connection between them is not in the language, it is in what was said before.

That is why reference is not resolved in the sky but at the base, by search, just like spelling mistakes. The base keeps the constellations of recent sentences, an *atlas* of the conversation. When the pilot meets a pronoun, it searches the atlas for a word star compatible with the pronoun's kind: "she" calls for a feminine singular name, "that one" for a thing already mentioned, "the other" for the second member of a pair of the same kind. It searches from the most recent sentence backwards, and looks first in the domains the verb of the current sentence expects (after "reads", an agent of the kind person). The star found, Ana's, is *borrowed*: it is drawn on the chart of the second sentence in place of the star of "she", and the chart now says "agent: Ana". If two stars fit equally well (two feminine candidates equally recent), the pilot does not choose: it fuses both into the agent's domain, with the "one of two" mark, so that the chart does not read them as "Ion and Petre". A following sentence, or the world, can close it: the world knows that a book does not read, the pilot does not, and need not.

The division is the same as everywhere in the story: the pilot proposes by form (the kind of word, recency, the expected domains), the world decides by facts. None of this requires the pilot to understand who Ana is.

The mechanism is precise and asks the pilot to learn nothing new: the pieces already exist (the catalogue, the domains, the fusing of candidates, the search). What is measured is how often it chooses well on real text, and how often it says it cannot choose when it truly cannot.

Why the word stars are random, and must stay that way

You might think learned stars would be better: "cat" next to "dog", "book" next to "novel". But in this story, closeness between word stars is interference. If "Ion" and "Petre" were close, the agent's domain could not tell them apart; the margin on which every diagnostic above rests would shrink. Random stars are unrelated to one another, and that is what makes each domain readable. Semantic closeness between words belongs elsewhere: to the way words are grouped into kinds (which decides their keys) and to the knowledge the world has about what a cat is. The pilot's job is to draw the right star in the right domain; it needs the stars to be distinct, not similar. This also settles spelling mistakes: a misspelled word is matched to its correct form *before* reading, through a search that can use letter patterns, and the star of the correct form, random like all the others, is the one drawn. Tolerance lives in the search, never in the stars.

9. Where it leads

9.1 Assistants allowed to say "I don't know"

Take a single field: a timetable, an inventory, a product catalogue, high-school physics once numbers and units have named stars of their own. Write its legend: the roles its questions can have, each with its star. Train the route book on questions from that field, paired with their correct constellations (a query, a search, a computed answer). Then run.

A user types. Word by word, the probe jumps and the pilot draws; you can literally show the frame filling in as they type, because each step is a few thousand turns of the dials and the whole sentence costs microseconds on an ordinary processor. When the sentence ends, the frame is read off the chart and handed to the database or the reasoning engine. The answer comes derived, and the derivation can be shown beside it: here is what I understood, here is what I asked, here is what came back.

When the chart does not match the legend, the assistant does not guess. A domain left unrecognised: “you didn’t say which product”. A crowded domain: “the March invoice or the April one?”. Stars left unexplained: “I have no information about that”. A word without a kind: “I don’t know the term X”. Each refusal is a distance read off the chart, not a hand-written policy; and each is a statement about the pilot’s own legend, not a verdict on the user.

Such an assistant is slower-witted than a chatbot. It makes no conversation, has no opinions, knows nothing outside the legend. In return, its answers are restricted to what the legend and the database contain, each comes with a margin, and each can be reproduced and verified. It can still be confidently wrong, drawing a word in the wrong domain with a clear margin; how often that happens is a number to be measured for each field, and the assistant must be judged by it. There is no language model in the loop, no GPU, no cloud call. It can run on a phone or in a browser, and it can afford to verify every message.

Messy input is handled in layers, and without pretence. Spelling mistakes are mostly absorbed by classical correction before reading, a search that takes the wrong form to the correct one; the star drawn is always that of the correct form. When a sentence is truly incoherent, a small language model can be set to rewrite it, but it does not have the last word. The rewrite is read too; every star recognised in the original must still be present; everything the rewrite added is marked as an assumption and put to the user as a question. The assistant does not claim to have verified a rewrite of something it did not understand. It limits what the rewrite can break, and asks.

9.2 A verifier beside the large models

The other direction. A large language model reads a complex document and proposes a formal reading: a summary, an extraction, a set of facts, a logical form. It knows far more language than we do. And, sometimes, it misses a sentence, swaps two roles, or silently adds a plausible detail.

We read the same document with our pilot, separately, and compare the constellations domain by domain. Four verdicts, not three. *Agreement*: the same star in the same domain; two separate readings agree, which means neither that the reading is true nor that their errors are independent, especially when both learned from texts produced by the same models. *Compatible, incomplete*: one says “Tuesday”, the other “Tuesday morning”; not a contradiction, a detail more. *Contradiction*: different stars in the same domain, neither refining the other; we say in which domain, not who is right. *Unverifiable*: our chart is uncertain, a domain unrecognised or crowded, or the sentence is outside the legend. The last verdict is the valuable one: a language model can be taught to abstain, but its

abstention is not tied to a structure a person can check; ours says in which domain and why.

For this we too must be trained on the field; not to match the large model's coverage, but to have our own constellation of the text to compare against. The part of a document we can confirm or contradict grows with training, and the part marked unverifiable shrinks. The verifier is honest at every point about which is which. In regulated fields, that honesty is worth more than one more point of accuracy.

9.3 Formalising a field for symbolic reasoning

The frame read off the chart is already a key-value structure, with levels wherever the sentence had sentences within sentences. Turning it into facts for a logic engine or into constraints for a solver is a mechanical rewrite *once the frame, with its operators, has a fixed semantics*: the agent's domain becomes the first argument of a predicate, levels become nesting of terms, a negation drawn as a role becomes a negated literal. The hard choices are made once, in the definition of the frame, not in the translation: what a quantifier means, how two mentions of the same entity are identified, how time and modality are treated, and which negation is translated (the classical negation of a formula is not the negation-as-failure of a logic program). The translation learns nothing, so it does not err the way learned steps do; but it also translates a wrong definition consistently.

The path from natural language to formal reasoning becomes: text, chart, frame, formal statement, reasoning engine, answer. Every link is inspectable. The whole loop can be checked against the world's own answer, and in our controlled worlds it is exact, including on levels never trained.

The logical operators themselves are stars in the legend. Negation is a star: the thing negated is drawn from a position that contains it, and its scope, what exactly is negated, is simply what was drawn that way: the whole sentence, or a single argument. "Ion does not give Ana the book" and "Ion does not give the book to anyone" draw different things, and the pilot reads the difference without a rule for either. Quantifiers are stars attached to arguments. Where a sentence is truly ambiguous, "every child reads a book", the chart holds both readings and, when read, shows both at nearly equal strength: the ambiguity is reported, not resolved by a guess, and the pilot was not trained on ambiguous sentences to do this. We observed it; we did not design it.

In a restricted field with a known schema, this formaliser is faster than any language model by orders of magnitude and at least as correct, with the decisive difference that it reports how sure it is, domain by domain, and can be held to account for that report. On open text, the realistic role is the one in 9.2: not to replace the large model's formalisation, but to verify and complete it.

9.4 A boundary that learns

Any system built from several models has seams, and at every seam something symbolic appears: a list of detected objects with positions and velocities, a set of extracted fields, a parsed command. It has to appear: people inspect the seam, the parts are trained

separately, the rules live there. The known weakness is that the seam's schema is drawn up by engineers in advance and loses everything they did not foresee; and the known reaction, training everything end to end in a single latent space, buys coverage at the price of a seam nobody can read.

Our chart is a seam of the first kind with some of the virtue of the second. Its domains, the roles, are still given by the world; but the way words reach them, and which words set beacons, is learned, not hand-written; new named stars can be added to the legend when the world calls for them, with only the route book to retrain; it is readable by construction; and it can keep, fused beside the domains, a residual for everything that does not yet have a name, so that what the schema did not foresee is blurred, not lost. The same chart can be filled from several senses at once, because a star is just coordinates and a domain is just a named star: a word, an object detected in an image and drawn from a position that encodes its location, an event from a log, a reading from a sensor. In fact, language was the hard case: in text, the key has to be guessed from the word. Wherever the key is given directly, by a motor command, by elapsed time, by a coordinate, the machinery applies with nothing to guess, which is precisely the setting Hawkins's theory of the cortex was conceived for in the first place.

9.5 A world model you can read

Where the world's constellation is the consequence of an event, not its description, learning to produce it from the sentence is a step towards learning the dynamics of a world: state plus event gives the next state. We have done this only on small worlds, and reading an event into a frame is not yet the same thing as predicting what follows from it; the claim here is a direction, not a result. Today's world models do this in a latent space nobody reads, and imagine the future by running a network forward, step after step, with errors that accumulate and nothing to check against. Our state is a chart with a legend. Wherever the world has rules, dates and deadlines, contracts, procedures, code, databases, "imagining the consequence" can be handed to a symbolic reasoning engine that works on the frame: exact, explainable, stable over any number of steps. Where the world has no written rules, the fall of a glass, the latent approach keeps its place, and a hybrid uses the frame as a skeleton and a latent residual for the rest.

10. Robotics: a semantic interlock

Robots increasingly receive commands in natural language, and the road from command to motor runs through a large model: a planner that turns "put the box on the top shelf" into a sequence of movements. The model is capable and opaque. When it errs, it errs in the physical world, and nobody sees the error before it happens. For the rest of the robot, industry has an old and good solution: the *interlock*, the device that physically cuts the power to a forbidden action, whatever the rest of the system has decided. What is missing is an interlock for *meaning*: something that knows what was asked and what was proposed, and blocks what is not permitted before the motion.

What it looks like

Between the planner and the actuators stands the pilot, with an action legend written for that robot: which action, on which object, where, at what speed or force, under which condition, at whose request. Every command from a person and every action proposed by the planner passes through it and becomes answers to these questions. What does not fit the legend does not exist for the interlock, hence cannot be executed: an action nobody foresaw has no way of being expressed. The frame that is read is translated mechanically into facts, and the safety policies are rules in the same language: no fast motion while a person is in zone B; no object of class “fragile” above speed V; any action on objects in list L requires confirmation. A rule engine decides, with a derivation that can be displayed, and only permitted frames reach the actuators.

What makes this more than a word filter is that the same legend also reads the *sensory world*. A detected object with its position, a person who entered a zone, an open door are facts with roles too (what, where, when), and they enter the same chart through a jump given directly by the sensor, with nothing to guess (9.4). The rule “do not move fast while someone is in zone B” is then checked on the same chart on which the command sits, in the same engine. The interlock sees the command and the context in the same form.

Why it is cheap, and why it can be hardware

All the pilot does is add angles on a few thousand dials and compare against a catalogue. A command costs microseconds on an ordinary processor; the route book fits in a megabyte; no accelerator is needed. With the sparse block codes of chapter 14, the operations become arithmetic on integer indices, exactly the kind of thing that goes into a microcontroller or an FPGA. This opens an architecture that functional safety knows well: the semantic interlock on a separate circuit, with its own power and its own clock, which receives the command and the context from the main computer and alone emits the *enable* signal to the actuators. The main computer may be as complex and as opaque as it likes; the device that gives permission is small, deterministic and readable line by line.

Determinism matters here more than anywhere. The same command gives the same chart, every time; there is no sampling, no temperature, no “creative” answer. Every block can be reproduced and explained: here is the frame that was read, here is the rule it violated. Every acceptance likewise. For the certification of a safety system, that is the difference between a component that can be analysed and one that must be taken on trust.

What it does when it is not sure

A robot’s default is rest. When the chart is uncertain, a domain with a small margin, a large residual, a word without a kind, the interlock neither guesses nor blocks blindly: it *stops and asks*, and it can say exactly what it understood: “I understood: move the box to aisle B, no speed specified. Confirm?”. The confirmation is a sentence too, read in the same legend. The confirmation dialogue is, in fact, the device’s natural interface: the person sees the frame before the motion, not after.

What decides how far this goes

The pieces are those of the whole document: the reader, the frame, the translation into rules, the diagnostics. The action legend for a particular robot and its rules are domain engineering, as for any safety system. The measure that decides how far this can go is the same as in chapter 16: the scope of negation on real commands ("don't leave the box here, take it there") and the confident-wrong rate, because an interlock blocks by construction only what it cannot express: a dangerous action can be perfectly expressible (deleting the wrong object, a sequence of individually permitted actions), and a misreading with a large margin remains possible. The interface restricts and checks; it does not guarantee. The correctness of the reading and the adequacy of the policies are measured separately.

11. Enterprise and highly regulated industries

Agentic systems: intent read without a language model

An enterprise agentic system receives requests in natural language ("move invoice 4471 to the sales department's cost centre and notify Popescu") and turns them into calls to software: the ERP, the CRM, the ticketing system, the calendar. Today the step from request to call is made by a large model, at every request, with its latency, its cost and its non-determinism, and with the risk that a request is understood otherwise than it was said and executed anyway.

For a fixed domain, the domain of a piece of enterprise software, with its entities, actions and parameters, the pilot makes this step on its own. The legend is the system's schema: which actions exist, on which objects, with which fields. The route book is trained once on requests from that domain, with their frames, produced by the annotation pipeline with large models (9.1) and audited on a sample. After that, a clear request becomes a complete frame in microseconds, on an ordinary processor, inside the company's network, with no external call: the *intent* is the action whose domain filled up, the *parameters* are the stars in the other domains, and what is missing is in plain sight. "Move invoice 4471 to the sales department's cost centre and notify Popescu": action move, object invoice 4471, destination the sales cost centre, notified Popescu; deadline, not given.

The large model does not disappear; it changes place. The architecture has two paths. The fast path: the request is read by the pilot; if the chart is sharp, all mandatory domains filled with a clear margin and a small residual, the frame goes straight to execution, after the policy check. The slow path: if the chart is uncertain, the request goes to the large model, which rephrases or completes it, and its result is read again by the pilot, because nothing reaches execution other than as a verified frame. The large model works on the hard tail of requests, not on all of them; its cost and latency are paid on a share of the requests, not on all; how large the share is, is measured per field, not assumed.

Three things come for free from this arrangement. *Authorisation* becomes a rule on frames: who may perform which action on which objects is written in the same language as the policies of chapter 16, and is checked before execution, not after. *The log* is a list of frames, not of texts: every executed action has next to it the request, the frame that was read, the margin and the rule that permitted it, readable by a person without tools. And *determinism*:

the same request gives the same frame, today and a year from now, which makes regression tests on understanding possible, not only on code.

Regulated industries: documents that must be read exactly

In pharmaceuticals, banking, aviation, energy, the documents that govern activity (standard operating procedures, laws, regulations, guidelines, audit reports) are, in essence, lists of facts with roles: who must do what, when, under which condition, with what evidence, with what exception, under which article. They are long, they change often, and they must be read exactly, because a missed obligation or a misunderstood deadline costs.

A legend for this kind of text has a few dozen questions: the obligated actor, the action, the object, the deadline, the condition, the exception, the required evidence, the normative reference. With it, the pilot turns a document into a list of frames, one per sentence that says something, each tied to its sentence, with the empty domains in plain sight and with the residual showing what did not fit the legend. From there, several operations that today are done by hand or by large models without verification become comparisons between charts:

- *The difference between two versions of a regulation*: not which words changed, but which obligations appeared, disappeared or changed their deadline or actor, domain by domain.
- *The conformity of a procedure to the norm*: the procedure and the regulation read in the same legend, then compared; an obligation in the regulation with no corresponding frame in the procedure is a visible gap, not an opinion.
- *Reliable data extraction* from reports, forms, notifications: every extracted value comes with its domain, its margin and its source sentence, and “not stated” is an answer, not an unexplained empty cell.
- *Verification of summaries and extractions made by large models* (9.2): confirmed, contradicted, unverifiable, frame by frame, with the exact place to check.

What an auditor or a regulator looks for in such a system is exactly what it has: the same input gives the same output; every answer is traceable to a sentence and to a line of the legend; the model is small and can be inspected in its entirety; the data does not leave the institution, because no external service is needed to read. The validation logic for computerised systems in regulated industries asks for components whose behaviour can be specified, tested and reproduced; a deterministic reader with a legend written by people fits that logic in a way that a large model, by its nature, cannot.

Where large models come in, and where they do not

We do not hide that large models are necessary. They write the first versions of the training frames, they paraphrase, they summarise open text, they handle the hard tail of requests and they propose formalisations of complex documents. Our role is the one defined throughout this document: the verifiable layer between language and action, the one that says what was understood, what was not, and where, and through which everything that reaches execution or the database must pass. The division is clean: the large model for

coverage, the pilot for verification. In an enterprise, this means lower inference costs, sub-millisecond latency on the fast path, and a log an auditor can read. In a regulated industry it means the possibility of using natural language where until now it was not allowed, because it could not be verified.

What sets the pace

For each domain, the legend and the route book: domain engineering, with the annotation pipeline already designed. The measures that set the pace are the verifier with injected errors on real text, calibration reported on every table, and the proportion of requests that stay on the fast path in a real domain. The direction is clear and the steps are incremental; what each step gains is measured in requests served without a large model and in errors caught before execution.

12. Further on: documents, novelty, bias, smoothing

A sentence is a short route. A document is a long one: many charts, linked by keys ("the contract mentioned above" is a key back to an earlier chart). Once documents are charts with legends, three things people care about turn out to be geometric.

Novelty. Read a scientific article against the legend of its field, where the named stars are the roles proper to the field: the problem addressed, the method used, the quantity measured, the result obtained. Most of the article's constellation falls in domains already occupied by known stars: a known method, a known problem. Novelty is the region where it does not fall like that: a star in a domain where the field had a different occupant, or stars at points the legend does not name at all. Not "it sounds new", but "it differs from the legend here, in this domain". The size and location of that region are the measure.

Bias. Two reports of the same event should draw the same constellation. When a source consistently draws the same stars in shifted domains, the one who acted becomes the one affected, the cause becomes a circumstance, that shift is an extra key, the same across many pairs of reports. A source's bias is the average direction in which its constellations deviate from those of the others, readable domain by domain. It does not require knowing what actually happened; it requires only several reports to compare, and the legend.

Smoothing. It is a concept from our own research: a bias that does not distort but omits. The inconvenient consequence is simply not mentioned. The answer is polished, diplomatic, and nothing in it is false. Methods that examine what is said cannot see it, because nothing that is said is wrong. In our story it is a bias of absence. An event's legend has a domain for its consequence and one for who bears it; in neutral reports of that kind of event, those domains are full most of the time. With a source that smooths, they are systematically empty. The route skirts a region of the sky. We propose to call it *frame-region avoidance*, and it can be measured without understanding the content of the consequence: it is enough to know that the domain exists in the legend, that it is usually full, and that here it is not.

All three are *candidate indicators*, not validated measures: a large remainder may be a misreading, not novelty; a deviation from the other sources may be a minority that is right,

not bias; an empty domain may be concision or missing information, not smoothing. Each requires controls (the question asked, the information available, the length, the relevance of the omitted consequence) before it earns the name of the phenomenon. And all are directions. They need legends at the level of events, not sentences, base rates per field for how often each domain gets filled, and the linking of references across sentences. It does not need the pilot to know the world. It needs only the shape of events, and the shape is what it learns.

13. For the technically inclined: the story, translated

Everything above was said in images. This chapter says the same things in the vocabulary of the field the images come from. Nothing here changes the picture; it names its parts. First the dictionary, then the details.

In the story	In the mathematics
a point in the sky; its coordinates (the code with thousands of dials)	a vector of D complex numbers of unit modulus (phases); D in the thousands
“close” / “unrelated”	high cosine similarity / near zero
two random points are unrelated	quasi-orthogonality of random vectors (concentration of measure)
the word’s star; the catalogue	the identity hypervector: random, fixed, unlearned; the codebook
to shift a point by a code	binding: element-wise multiplication = addition of phases; isometry, invertible, commutative
the inverse key; to shift back	multiplication by the conjugate (unbinding)
the pilot	the model: the learned parameters plus the fixed reading procedure; contains no facts about the world
the probe; the reported position	the address state A_t: the product of the displacement vectors used up to t
the key of a kind of word; the route book	the learned displacement vectors, per category
drawing the word’s star shifted by the probe’s position	binding of the identity with the current address: A_t (x) f_w
the sentence’s chart; the constellation; to fuse	the state S = bundle (unnormalised sum) of the deposits; compared by cosine
the legend; a role’s named star	the role vectors R_r, given by the world
a role’s domain	the set {R_r (x) f : f in the codebook}; an isometric copy of the codebook
“Ion in the agent’s domain”	R_agent (x) f_Ion

In the story	In the mathematics
the scanner	cleanup: the nearest neighbour in the codebook, by cosine; margin = the difference between the top two
the “reported” star; the shifted origin; the level	the role vector R_rep; nesting = binding with R_rep d times
beacons with relative keys, held aside, gates	the chain code with leftward re-placement; gates = learned sigmoids
the three drawings	bundle of address codes: path, bigram, chain
the world’s constellation	S_world = sum of R_r (x) f_filler, with the operators as roles
unexplained stars	the reconstruction residual
the “something was drawn here” marker	a fixed vector fused with every filler
the paths form a grid, not a tree	commutativity of binding: the address is a multiset of roles

The family

The story is a Vector Symbolic Architecture (VSA), also called hyperdimensional computing, a line of work that goes back to Plate’s holographic reduced representations (1995), Kanerva’s hyperdimensional computing (2009) and Gayler’s multiply-add-permute. Of its variants we use the complex one with unit-modulus components, which Plate introduced and which Frady, Kleyko and Sommer developed into *fractional power encoding*. The two operations, binding and bundling, are the primitives of any VSA; cleanup is nearest-neighbour retrieval. The quasi-orthogonality of random vectors is the concentration-of-measure fact that gives the family its power: the expected cosine between two random unit-phase vectors is zero, with variance of order one over D.

Atoms and compounds

A word’s identity is an atomic hypervector. The role vectors are atomic too, drawn once and fixed. A role-filler pair is the binding of the two; a frame is the bundle of its pairs, the standard VSA encoding of a structure, due to Plate and, before him, to Smolensky’s tensor products. Nesting is binding with the “reported” role vector; reading a role at depth d means unbinding with the conjugate of that vector d times, then with the conjugate of the role, then cleanup. Composition at an unseen depth is the associativity of this algebra.

What the model computes

For the words w_1 … w_T, each with a fixed identity f_w and a category c_w, the state is S = sum over t of A_t (x) f_{w_t}, where A_t is the address of position t: a bundle of codes, each a product of learned phase vectors. The *path* code: the product of the category vectors of all the words up to t (path integration). The *bigram* code: a vector indexed by the previous category and the current one. The *chain* code: the address taken from an expectation; a

word whose category has its gate open (a learned sigmoid) emits, for each category, an expectation at its own address bound with a learned relative vector; a later word of that category takes the most recent unconsumed expectation; a word that arrives before any expectation is held and placed by the first emitter that follows. For learning the gates, a differentiable relaxation weights each candidate address by the gate's sigmoid, so that gates and phases train with the same gradient.

Because element-wise multiplication commutes, each address is a multiset of vectors, not a sequence: role paths that differ only in order bind to the same address. The non-commutative variants (permutation-based binding, level-indexed roles) are the known alternatives and give up free composition across depths. The worlds so far had no predicate role and no event identifiers; both are additions that do not change the architecture.

All codes are learned per category, never per word: a few hundred phase vectors plus one gate scalar per category. The categories come from outside the model (given, on synthetic worlds; from clustering a skip-gram embedding, on real text). We have shown that the clusterings induced by its own displacements coincide with those of a simple skip-gram on the same sentences, and we do not claim them as emergent; the gates are not recoverable from co-occurrence and are the part that emerges.

The world and the objective

The world gives, for each sentence, S_world = sum over roles of R_r (x) f_filler, with nesting through R_rep and the operators (negation, quantifier scope) bound as roles around their scope. The identities are shared between model and world; the role vectors are the legend. The objective is maximising the cosine between S and S_world. It is generative: the model produces the world's vector through a fixed isometry, not through a learned function. We have shown that this fixedness is what forces convergence: a learned associative readout, even of limited rank, takes distinct states to the same target without making them equal, and then does not generalise to new identities and unseen depths, exactly like a small transformer. Training is Adam on the phases and on the gate logits; minutes on a GPU, feasible on CPU.

Reading back and the diagnostics

Decoding role r at depth d: S bound with the conjugate of R_rep d times and with the conjugate of R_r, then cosine with each candidate; the argmax is the filler, the difference from the second is the margin; the depth is estimated by trying each d. The frame is the set of these readings; conversion to Prolog or SMT-LIB is a syntactic rewrite of the frame, given a fixed semantics of the operators (classical negation in SMT; in Prolog, negation as failure, with its caveats).

The diagnostics of chapter 8 come from the same readings, with one easily overlooked qualification: unbinding is an isometry, so the norm of what comes back from a role says nothing about occupancy. What is available: the best cosine and the margin (recognition); the reconstruction residual after scale fitting, relative to the norm and to the world's background (the unexplained matter); the position at which the margins collapse (the

broken key). Telling an absent filler from an unknown one requires the occupancy marker. S is a plain, unnormalised sum; comparisons are by cosine.

What is standard, what is ours

Standard: the algebra, role-filler encoding, nesting through binding, cleanup, the capacity results, fractional power encoding and the reading of grid cells through it (Frady, Kleyko, Sommer), which is the link to Hawkins' theory.

Ours, and tested: that addresses can be *learned* as products of per-category phase vectors from a generative objective on a world with explicit roles, so that paraphrases bind their fillers to the same roles without paraphrase labels; that a bundle of redundant address codes makes this robust to unseen constructions; that addressing through expectations with leftward re-placement gives clean addresses and composes one level further than it was trained; that the gates emerge by gradient and coincide across worlds and with real text; that logical operators bound as roles around their scope are read correctly, scope included, without rules; and that the same state gives, through margins and the reconstruction residual, a usable account of what the reader did not understand.

14. What the sky can be made of: representation variants, sparse codes and the brain

The story has been told in a single material: points with coordinates on thousands of dials, that is, vectors of phases. It is not the only possible material. The family of representations we belong to, vector-symbolic architectures, has built the same three operations, shifting, fusing, recognising, in several ways, and the choice among them is an engineering decision with consequences for cost, for hardware and for closeness to what is known about the brain. This chapter says what the variants are, why we chose ours, what sparse codes would bring, why the brain's code is believed to be sparse, and, above all, which part of what we built depends on the material and which part does not.

The same three operations, several materials

All the story asks of a material is this: points chosen at random must be unrelated to one another; several points must be fusable into one from which each remains recognisable; and there must be a shifting operation that preserves shape and undoes exactly. Each family achieves these differently.

- *Sign vectors* (+1 and −1 over thousands of positions): shifting is position-wise multiplication, which is its own inverse; fusing is a majority vote at each position. Simple, binary, cheap; shifting is exact only approximately after fusing, because the vote rounds.
- *Real vectors with circular convolution* (Plate's holographic reduced representations): shifting is convolution, the inverse is approximate (correlation), fusing is addition. Historically the first complete variant; the approximate inverse costs precision at large depths.
- *Complex phases* (our variant, also called holographic in the frequency domain): each position is an angle; shifting is adding angles, the inverse is subtracting them,

exactly; fusing is adding arrows. It is Plate's convolution written in the Fourier domain, where it becomes exact and cheap.

- *Sparse block codes*: each position is a block of L units of which one is active; shifting moves the active unit within the block; fusing is superposition followed by a competition within the block. The sparse equivalent of phases, described below.
- *Sparse binary codes with thinning*: few active units out of very many; fusing requires an operation that preserves sparsity; shifting is a context-dependent operation. The closest to the sparse representations of neuroscience, the farthest from gradient learning.
- *Binding by permutations or matrices*: shifting is a permutation of positions or a multiplication by a matrix; it does not commute, so it preserves order, at the price discussed in chapter 8 under "Order".

What they all share: separation by randomness, superposition, invertibility. Where they differ: how exact the inverse is, how many things fit in a chart at the same number of positions, what an operation costs, and how easily a shift can be learned by gradient.

Why we chose phases

Four reasons, in order of importance. The inverse is exact, so reading a domain at any depth accumulates no material error, only the noise of superposition. Shifts are continuous, so the route book is learned by gradient, step by step, which made the whole of chapter 6 possible. The same encoding serves continuous quantities too: a number, a position, a moment are phases with an angle proportional to the magnitude, and "40" and "41" are nearby stars without being identical. And, fourth, phases are exactly the mathematical model by which the hyperdimensional computing literature has described the path integration of grid cells, which ties the story directly to the theory of the next chapter.

The price is that phases are dense: every dial always has a value, and a chart is a string of thousands of real numbers. On an ordinary processor this does not matter; on very simple hardware, or as a model of cortex, it does.

The sparse code: the same sky, written in neurons

The dense phase representation has a sparse equivalent, and the equivalence is exact, not an analogy: a dial with L positions is a block of L units of which exactly one is active, the one at the dial's angle. Shifting becomes moving the active unit within the block (addition modulo L); fusing becomes superposition, several active units in a block, followed by a competition that keeps the k most active; the scanner stays the same. The family is known (sparse block codes), and the equivalence with phases goes through the Fourier transform.

What it would bring. Integer index arithmetic instead of complex numbers, hence direct implementation on microcontrollers or programmable circuits, exactly what the interlock of chapter 10 asks for. Memory of the order of bits per dial (log L bits) instead of tens. A natural normalisation: the competition within the block is precisely what the dense chart lacks, which at many stars slides towards the "middle"; and, interestingly, the competition preserves unions of hypotheses: two active units in a block mean "either one or the other",

that is, exactly the union of hypotheses of Hawkins' theory, arising by itself. And a sparsity of the order of a percent, as in cortex.

What it would cost. Angles are quantised to L positions, so continuous quantities have a granularity of 360/L; at L in the tens or hundreds this probably does not hurt, but it is one more parameter. Training stays on continuous phases, with quantisation at the end; how much is lost is measured, and it is one of the planned experiments. Capacity is comparable in order of magnitude, but the details vary with L and k.

Why the brain's code is believed to be sparse

Three lines of evidence, with their reservations. First, the energy budget: a firing neuron is expensive, and the classical estimates show that cortex can afford only a small fraction of neurons active at the same time, of the order of a few percent; a dense code would be metabolically unsustainable. Second, sparse coding as a computational principle: the work of Olshausen and Field showed that a system that learns to represent natural images with as few units active at once as possible rediscovers by itself the receptive fields observed in visual cortex; it is one of the most reproduced results in computational neuroscience. Third, structure: grid-cell modules work as attractors in which a single "bump" of cells is active, whose position encodes the phase, that is, exactly a sparse block with one active unit; and the theory of the next chapter puts sparse representations at the foundation of the whole neocortex.

The reservations, just as clearly: there are areas and regimes with dense activity; the debate about how much firing rate matters against the timing of spikes is not closed; and "sparse" is a finding about activity, not a proof that the algebra the brain uses is one of those above. So we say "believed, with consistent evidence", not "known".

Where the innovation is, and where the engineering is

The algebra, in any of its variants, is standard and thirty years old; we claim nothing of it. What we built sits *above* the material and does not depend on it: the displacements learned per kind of word, the three drawings of the same word, the gates that decide who puts beacons, the world as training target, operators as roles around their scope, reading with margin and remainder. Each of these is written the same way on phases, on sparse blocks or on signs; it would be the same story, with another sky. For whoever evaluates the project, that is an argument of robustness: the results do not hang on a choice of material, and the material can change when the hardware target requires it.

The engineering is the choice of material for the target: the number of dials, L and k for the sparse code, the quantisation, the mapping onto a circuit. And the future research is here too, with precise questions: whether training directly on the sparse code (not only quantising at the end) preserves the results; whether the competition within the block gives the union of hypotheses with the same precision as dense superposition; whether non-commutative binding can be added to the operators alone, keeping free composition for the rest; and how much of a chart's capacity is recovered by structures of charts instead of more dials. None of these changes the story; each changes how cheaply, and how close to the brain, it can be told.

15. The parallel with *A Thousand Brains*

The story above did not come out of nowhere. The starting point was Jeff Hawkins' theory of the neocortex, set out in *A Thousand Brains* (2021) and in the Numenta group's papers before it. We owe a fair side-by-side: what we took, what we translated, what we have in another form and what we do not have at all. It is also instructive, because every piece of the story has a counterpart in the theory, and the differences say exactly what kind of thing we have built.

What the theory says, in five sentences

The neocortex is made of tens of thousands of *cortical columns*, all with the same structure. Each column learns complete models of objects in the world, not fragments. A model is a *reference frame* anchored on the object, in which features are stored at *locations*: to know a cup is to know what is felt at each spot on it. The current location is held by cells resembling the grid cells of the hippocampus and is updated by *movement*: when the finger moves, the location moves with it, before any sensation arrives (path integration). The columns, each with partial evidence, *vote* through long-range connections and converge quickly on a single object; as long as they do not converge, they hold open a union of hypotheses. And, the boldest claim: the same mechanism, reference frames plus movement, also serves for concepts and language, where "movement" is abstract.

The dictionary

In Hawkins' theory	In the story	In the mathematics
an object's reference frame	the sky, with the legend as anchors	the space of phase vectors; the role vectors
a location in the frame; grid cells	a point in the sky; each dial is the phase of one grid module	a vector of phases (fractional power encoding)
movement; path integration	the probe's jump; position = the sum of the keys	binding of addresses; the product of the displacements
a feature stored at a location	the word's star shifted by the probe's position	A_t (x) f_w
the model of an object	an event's constellation	the state S
object recognition	comparing constellations; the frame read through the legend	cosine with S_world; unbinding and cleanup
displacement cells; object within object	the "reported" star; the legend shifted to another origin	binding with R_rep; nesting
predicting the next sensation	the beacons: where the next word of a given kind will fall	the chain code with relative vectors
voting between columns	the three drawings of the	bundle of address codes;

In Hawkins' theory	In the story	In the mathematics
	same word, fused; the scanner	cleanup with margin
the union of hypotheses	a star drawn between two domains; the ambiguity held open	the superposition of two addresses in S
surprise; what does not fit the model	the stars outside the legend	the reconstruction residual
thousands of columns	a single pilot	a single model

What we took literally

Two things are taken as they are, not as metaphor. First, *location as phase*. Grid cells encode a position as a set of phases, one per module, and movement adds phases; the fractional power encoding we use is exactly this mechanism written as vectors, and it was proposed in the hyperdimensional computing literature precisely as a model of grid cells and path integration. The dials in the story correspond, in the mathematical model, to grid modules; it is a formal correspondence, not a claim about biology. Second, *knowledge as features at locations*. A sentence's chart does not store the words but the words-at-positions, and two sentences are the same object when they have the same features at the same locations, whatever the sensor's path. That is Hawkins' definition of an object, applied to events.

The third borrowing is the one we discovered only along the way: *composition by displacement*. Hawkins proposes "displacement cells" that hold the vector between two locations, so that one object can be placed in another's frame: the logo on the cup is the logo's frame, placed at a displacement within the cup's frame. The reported sentence in "Maria says that…" is exactly that: the inner sentence's frame, placed at the "reported" displacement within the outer one's frame. The fact that the pilot reads three levels after learning two is the property the theory predicts for displacements: a displacement applies any number of times.

Where the vote is

The best question anyone has asked us about the story was "where is the vote?". In the theory, the vote is between columns: each column has a hypothesis about which object it is sensing, the hypotheses are sent to one another, and the set of candidates narrows until one remains. We have no columns; we have a single pilot. But the vote exists, in two smaller places.

The first: *the three drawings*. The pilot has three ways of finding out where a word is, from three different sources of evidence, with different ways of going wrong: the path travelled (the sum of all the keys since the start of the sentence), the local transition (what kind of word came just before) and the expectation opened by an earlier word (the beacon). Each is a "column" that says "the word is here". We do not let them argue; we fuse them. Fusing is a vote by superposition: where the three agree, the arrows add up and the star is sharp;

where one is wrong, the other two cover for it and the star is only slightly blurred. It is a vote without debate, in a single round, and that is exactly why it is cheap.

The second: *the scanner*. When the chart is shifted back by a named star, what results is compared with all the stars in the catalogue at once, and the closest wins, with a margin. It is a vote between candidates, not between columns, but it has the same form: all compete, one comes out, and the difference from the second says how convinced the vote is. And when none comes out, when two candidates are equally close, the pilot does not force it; it reports both. That is the theory's *union of hypotheses*, held open until evidence arrives that closes it: that is how we read "every child reads a book" with both meanings at once, and that is how the first word sits halfway in two domains until the verb arrives.

Why only three

Because three is the number of *different* sources of evidence we found, not a chosen number. A vote is useful only if the voters go wrong in different ways; three copies of the same computation add nothing. The path travelled goes wrong when one key in the sequence is wrong and sends everything after it astray; the local transition almost never goes wrong, but it is coarse, because it knows only what came just before; the beacon is precise, but it is missing when the construction is new and nobody announced it. The three cover for one another, and the measurement shows how much: never-before-seen constructions are read nine out of ten with all three, six out of ten with one alone.

We also tried a fourth source: the word's place in the sentence, third, fifth. It brought a little extra on new constructions, and yet we dropped it. The argument of principle is that it ties the reading to the word's place in the sequence, that is, to exactly the thing paraphrases change; and the decision is re-measured with both variants on the same tests. A fifth source would again have to be a different way of going wrong, and we have not found one yet. If one appears, it is added as a new arrow to the fusing, with nothing else changing: that is the advantage of voting by superposition.

There is also a broader answer to "why not thousands". In the theory, the thousands of columns each receive a different piece of the world: a different finger, a different patch of retina. In text, the input is a single sequence of words, and there are no thousands of independent sensors to feel it from different angles. The truly "thousand brains" version would have many pilots reading the same text with the catalogue of letters, with that of words, with that of word groups, or several senses at once, as 9.4 says, and they would vote among themselves. It is a step still to take; the three drawings are its smallest form, within a single pilot.

Where we part ways, and why

Movement is guessed, not given. In the theory, the location is updated from the sensor's movement, which the brain knows because it commanded it. In text, nobody tells us how far "we move" at each word; the jump has to be inferred from the kind of word, and that is all the pilot learns. Language is the theory's hard case, not its easy one; wherever movement is given directly, by a sensor, by a motor command, by time, the machinery applies without the learned part.

Learning is from the world, not from prediction. Hawkins' columns learn by predicting the next sensation, with no teacher. Our pilot learns from the world's constellation: it is told what the chart must look like, and adjusts its keys to get there. We tried the zero-teacher variant, predicting the next word, and the domains disappeared: nothing forced the pilot to separate the roles. We do not claim that it is otherwise in the brain; we claim that in text without a world, prediction alone did not give us reference frames with roles. It is a difference of regime, not of mechanism, and we state it as such.

The frame is given, not learned. In Hawkins, the column itself learns the object's frame. In ours, the legend's named stars come from the world; what the pilot learns is the path to them and who opens frames (the gates). We checked that the kinds of words do not arise on their own from our objective, and we withdrew the claim to the contrary; the gates, on the other hand, do arise.

A single forward pass, with no memory between sentences. The theory has columns that integrate over time and revisit their hypotheses; the pilot goes forward, one sentence, one chart, and forgets. Structure announced afterwards and pronoun reference are exactly the places where the lack shows.

The substrate. The theory's representations are sparse and binary; ours are dense, phases on thousands of dials. The properties that matter for the story (two random points are unrelated, superposition holds several things, shifting preserves shape) are common to the two families; the capacity details differ.

What remains, once the differences are subtracted, is the claim we tested and which holds: a reference frame with locations given by movement and features deposited at locations, with composition by displacement and voting by superposition, is enough for different sentences about the same event to arrive at the same chart, on constructed worlds and on a real set of questions, and for the chart to say where it did not arrive. The rest of the theory, the thousands of columns, learning without a teacher, learned frames, is what comes next, and it is the list on the table.

16. Alignment and the problems around it: what can be said in engineering terms

By *alignment* we mean here something modest and verifiable: a system that receives requests in language should do what it was asked, only what is permitted, and it should be plainly visible when it has not understood or has stepped outside what is permitted. We do not propose a solution to the alignment of large language models; we do not touch their internals and do not claim we could. What we have is something else: a *verifiable interface* between language and action, and a few measurements that become possible as soon as meaning is defined as in 1.1, a set of answers to questions known in advance. For each problem, this chapter says what mechanism we have; what is built and what is in progress for each is read in the project's status file (https://github.com/ml-circuits-research/vsa-semantic-space/blob/main/STATUS.md).

The basic idea: the legend as a boundary

A system that acts in the world receives requests in natural language and produces actions: a query, a command, an answer, a change in a database. Most alignment problems live at this crossing: the request is understood otherwise than it was said, the action is other than the one requested, or the answer sounds right and is false. The engineering proposal is that the crossing should not be made directly. Every request and every action proposed by the large model passes through the pilot, who turns it into answers to the questions of an *action legend*: which action? on what? with which parameters? under which condition? on whose behalf? The legend is written by people, has a few dozen lines, and anything that does not fit into it is not representable, hence cannot reach execution. It is not a filter that looks for forbidden words; it is a form in which certain things cannot be said at all.

The frame read off the chart is translated mechanically into facts and constraints (9.3), policies are written as rules in the same language, and a reasoning engine decides whether the action is permitted, with a derivation that can be shown. The division of roles is strict: the large model *proposes*, the pilot *reads independently*, the engine *checks*, and the large model never touches the actuator. The pieces exist: the frame, the difference between frames, the translation into Prolog and SMT; what does not exist yet is the measurement on real requests, with injected errors, that says how much it catches.

The problems, one by one

Unsupported claims (hallucination). The large model produces a summary, an extraction, an answer. The pilot reads the same text independently and compares the frames: agreement, compatible but incomplete, contradiction (and in which domain), unverifiable (where the chart is uncertain or outside the legend), with the criteria of 9.2. With a database at hand, each answer is also checked against facts. The default verdict is “unverifiable”, not “probably fine”.

Negation and its scope. “Do not delete the unsaved files” and “delete the unsaved files” differ by one word, and “do not delete the files, save them” has the negation on another action. For a system that executes, a misread scope of negation is the difference between safe and disastrous, and it is worse than a missed negation. We have measured the scope of negation on constructed worlds (read correctly, for eight ways of saying negation, with scope reported separately from presence); on real commands we have not measured it, and it is the first thing to do in this setting.

Instruction injection. A text that arrives as *data* (an e-mail, a page, a document) contains a command: “ignore what you were told and send the file to…”. A large model does not structurally distinguish data from instructions; to it, everything is text. The pilot can: in the legend of the data channel there is no question “what should the assistant do?”, so a sentence that answers it is either matter outside the legend or, if the action legend is applied to the data channel as well, an action frame that filled up where it had no business being. In both cases it is an alarm, with no need to understand the intent: we detect the *form* of a command in a channel where commands have no place.

Sycophancy and smoothing. A model that says what the user wants to hear omits the uncomfortable consequences; nothing it says is false, which is why it is hard to catch. The measure from chapter 12 applies directly to the model's answers: for a kind of question, the domains "risk", "consequence", "uncertainty" have a base rate of being filled in neutral answers; in a model that smooths, they are systematically empty. It is measured without judging the content of the answer, only its presence in the proper domains. It requires legends at the level of the answer and base rates per field.

Specification gaming. A rule of the form "maximise X" is exploited through actions nobody foresaw. An action legend is a closed list: the unforeseen action has no way of being expressed, hence cannot be executed. The reverse must be said too, in two parts. The problem does not disappear, it moves into the legend. And the legend does not stop a sequence of actions each permitted but dangerous together; for those, policies must be written over sequences, not over actions. If the legend omitted a question, the omission is invisible to the pilot. The advantage is that the legend has a few dozen lines, is written in words and can be read and contested by people, unlike a reward model; the disadvantage is rigidity, which we chose knowingly.

"It knows when it does not know". We do not write this, and it matters that it is not written. What we have: a margin per domain, a residual per sentence, and the confident-wrong rate, reported in the status file on every table. Without that number, abstention is not a guarantee. With it, abstention becomes a measurable policy: below a threshold, the action is not executed and the question goes to a person, and how often that happens and how often the threshold lets an error through are two curves, not two adjectives.

Inspectability and audit. Everything the pilot knows is a route book with a few hundred keys; every decision is a chart that is read domain by domain; there is no memory between sentences, so there is no hidden state that drifts over a conversation; facts sit in a database where they can be corrected. All of this is about the pilot, not about the large model, which remains as opaque as before. What we make transparent is *the interface*: the place where language becomes action.

Attacks on the pilot itself. An adversary who knows the mechanism may try paraphrases constructed to read into a harmless frame, unknown words placed deliberately to break a key, crowded sentences meant to drown a domain. Everything that breaks the reading lowers the margin or raises the residual and falls into "unverifiable", which is the safe default; but an attacker who knows the route book can construct sentences with a large margin and a wrong frame, and the rate at which that succeeds must be measured, with an automated attacker (a large model set to generate adversarial paraphrases), not assumed.

What it does not solve

It says nothing about the internal objectives of large models and does not change them. It does not work on open domains, without a legend. A wrong or incomplete legend gives a wrong or incomplete check, confidently. And everything above rests on a number reported in the status file, the confident-wrong rate; it sets the thresholds, not the architecture.

Summary

Problem	Mechanism	What is measured
unsupported claims	independent reading, frame difference, default verdict "unverifiable"	precision and recall on injected errors
negation and its scope	operators as roles; scope reported separately from presence	scope on real commands; confident-wrong rate
instruction injection	an action frame appearing in a data channel; matter outside the legend	true and false alarms on a data channel
sycophancy, smoothing	risk/consequence domains systematically empty against the base rate	fill rate against the base rate
specification gaming	closed action legend; policies as rules; an engine that decides	legend coverage; moves the problem into the legend
abstention	margin, residual, confident-wrong rate, threshold	the accuracy–coverage curve
audit	small route book, readable chart, no state between sentences, external facts	readability of the log, on a sample
attacks on the pilot	default "unverifiable"; success rate with an automated attacker	the attacker's success rate

17. What it is, and what it is not

The story of the sky, the probe and the pilot is an image; this chapter sets it aside and says what kind of system this is, what it compares against, and where its limits are, in the terms in which it would be discussed in a paper.

What it is

It is a *vector-symbolic reader* of sentences: a learned addressing scheme, laid over a fixed algebra. The algebra is that of phase-based vector symbolic architectures (binding by element-wise multiplication, bundling by addition, cleanup by nearest neighbour); the word identities and the role vectors are random and fixed; what is learned are the displacement vectors per word category, the relative vectors of the expectations and one gate per category, in total on the order of a megabyte. The output is a state that encodes a role-filler frame, nested by repeated binding, and that is read back by unbinding and cleanup, with a margin per role and a reconstruction residual per sentence. The training objective is the match, by cosine, with a target state built by a world with explicit roles; the

supervision is structural and per sentence, with roles given in advance, and contains no paraphrase labels.

As a type of system, it is a frame-based semantic parser (a *frame parser*) with a geometric particularity: the state is not produced by a decoder but composed algebraically, so that composition at unseen depths, the handling of new vocabulary and the uncertainty diagnostics follow from the algebra, not from learning. It holds no facts: the facts live in a database or a reasoning engine, and the frame translates mechanically into Prolog facts or SMT constraints.

What it compares against

Against transformers and language models. There is no next-word prediction objective and there is no generation. The fair comparison is with a small transformer trained on the same input and the same targets: on constructed worlds, at a nesting depth not encountered in training, the reader decodes the inner roles completely, while the transformer is at chance level; on WikiSQL, in a setting modified from the official protocol (the question alone, without the table schema; the selected column is predicted, not the executed query), the selected column is 0.82 against 0.54 for the transformer with the same input (0.65 with a ten times larger budget) and 0.50 for a bag-of-words logistic regression, at all training-set sizes from one thousand examples upward (the table by size, with seeds and intervals, is in the repository). In the other direction, where the decision is lexical (the aggregator in the same queries), the neural classifiers win (0.89), and whole-query accuracy (0.37) is not directly comparable with the figures of specialised systems, which see the table schema and report other metrics; it is far from them. The claim being made is therefore precise: on the same input, without external knowledge, with little data, the reader learns the role structure better and generalises compositionally; it is not a competitor on open text and does not claim to be.

Against semantic parsers with the same supervision (semantic role labelling, frame parsing). The supervision is of the same kind: structured targets per sentence. The differences are the representation (a composable vector state, not a tree produced by a decoder), the vocabulary (random identities plus category, hence new words without retraining) and the reading of uncertainty (margin, residual, abstention). The direct comparison with a classical parser trained on exactly the same targets, one classifier per role, is in the measurement plan; until then, we compare on properties, not on numbers.

Against tensor and holographic representations (Smolensky, Plate, Kanerva, the fractional power encoding of Frady, Kleyko and Sommer). Encoding frames by role-filler binding and bundling is standard and thirty years old. What is new here is that the addresses are *learned* from the generative objective, that they are redundant (three address codes fused), that addressing through expectations with the re-placement of early words gives clean addresses and composes one level further than it was trained, and that the gates (which categories open frames) emerge by gradient and coincide across worlds and with real text. We also checked what does *not* emerge: the word categories coincide with those given by a simple skip-gram on the same sentences and are not claimed.

Against the Thousand Brains theory, chapter 15: reference frames with locations given by movement, features at locations, composition by displacement, voting by superposition; no thousands of columns, no unsupervised learning, with the frame given by the world.

Against latent world models (Dreamer, JEPA): the same formal task, state plus event gives the next state, with a readable state instead of a latent one; a direction, not a result.

What it is not

It is not a language model; it is not a knowledge base; it is not unsupervised; it is not a replacement for large models on open text. Its realistic place is beside a large model, as the part that produces a verifiable frame and says what it could not place.

What could change the message

It is only fair that we say which future measurements could move the scientific message of this book, and in which direction. Three can. *Calibration*: if the confident-wrong rate comes out high and does not fall with the known improvements, "it says where it did not understand" becomes "it reports uncertainty, but poorly calibrated", and the chapters on verification, robotics and alignment must be rewritten accordingly; it is the measurement with the highest stakes. *Real text with free paraphrases, against a baseline with the same supervision*: if a classical per-role classifier, trained on exactly the same targets, matches the reader on real text, the message moves from "learns the structure better" to "comparable in accuracy, with composition at unseen depths and with diagnostics"; the uniqueness stays with composition and with reading back, not with the figures. *Quantifiers*: if even the world correction does not bring them in line with the other operators, "operators are roles, read without rules" acquires a declared boundary.

What cannot change: the mathematical facts (isometry, commutativity, the unrelatedness of random points, the scaling of capacity) and the results already measured on constructed worlds. And a few measurements can only improve things: the non-commutative remedies for order, the occupancy marker and the predicate, pronouns, the detection of a missing question, the sparse code. The thesis, meaning as answers to a legend's questions, reached by a learned route, rests on what cannot change.

Limits

The structural limits: the capacity of a state (tens to hundreds of pairs at a few thousand dimensions; long texts are structures of states); the commutativity of binding (the order of two operators is retained only when something else differs; the non-commutative remedies give up free composition and are to be tested); forward-only reading (structure announced after its scope cannot be placed); a new word without context receives a random category.

The results at the time of writing come from constructed worlds, with five seeds, and from a single real dataset. What is measured, what is in progress and with which criterion is kept up to date in the project's status file: https://github.com/ml-circuits-research/vsa-semantic-space/blob/main/STATUS.md. This document does not change with every experiment; the file does.